Harness Engineering

A Practical Guide

Javier Sandino

Co-authored with Claude Sonnet 4.6

Last modified: September 7, 2026

A Practical Guide to Harness Engineering

Python-flavored. Project-agnostic. Opinionated where it matters.

What this guide is

A working reference for engineers building software with AI coding agents — Claude Code, Cursor, Aider, Codex, or whatever comes next. It distills the discipline of *harness engineering* — designing the environment around an agent so it produces reliable, maintainable software at scale — into patterns you can apply project-by-project.

The guide assumes Python tooling for examples (Ruff, pytest, pre-commit) but the principles apply to any stack. It assumes you have a working AI coding agent and have written code with it; it does not teach prompt engineering or model selection.

What this guide is not

- A list of prompts to copy-paste
- A defense or critique of AI coding agents as a category
- A reference for any specific model, vendor, or product feature set
- A replacement for reading code

How to use it

Read Heuristics first. It's one page. The rest of the guide expands on those twelve ideas, and you should be able to come back to that page and recognize every line as a compressed version of something you now understand.

Read Part I to get the mental model. After that, the parts are independent — read in order, or jump to whichever section matches the harness work you're about to do.

Contents

Heuristics 3

Part I — Foundations 4
1. What harness engineering is, and what it isn't . . . 4
2. The agent vs. the harness: a mental model . . . 4
3. The four primitives . . . 5
4. What good looks like . . . 6

Part II — Context architecture 7
5. The progressive disclosure pattern . . . 7
6. System of engagement vs. system of record . . . 8
7. Architecture Decision Records . . . 9
8. Anti-pattern: the monolithic context file . . . 11

Part III — Mechanical constraints 12
9. Why prose rules drift, and what to do about it . . . 12
10. Layered enforcement . . . 13
11. Pre-commit as the enforcement substrate . . . 14
12. Writing remediation-grade error messages . . . 16
13. Anti-pattern: noisy checks . . . 19

Part IV — Agent capabilities (skills) 21
14. Skills vs. docs: when each is the right tool . . . 21
15. Writing a skill that auto-triggers correctly . . . 22
16. Anti-pattern: skill sprawl . . . 25

Part V — Feedback loops 27
17. Research-plan-execute-verify as a discipline . . . 27
18. Sub-agents as context firewalls . . . 28
19. Designing loops you can watch . . . 30
20. The hybrid autonomy spectrum . . . 32

Part VI — Operating discipline 34
21. Handoffs between sessions and tools . . . 34
22. Keeping the harness evolving instead of drifting . . . 35
23. When to add a check vs. a rule vs. accept the failure . . . 37
24. The cost picture . . . 38

Appendix A — Reference toolchain (Python) 40

Appendix B — Templates 47

Appendix C — Further reading 53

Heuristics

Harness-engineering ideas worth remembering — keep them somewhere visible.

---

1. map vs. territory — `CLAUDE.md` routes; `docs/` knows. Don't conflate the system of engagement with the system of record.

2. progressive disclosure — load only what the current task needs. Context isn't free, and more isn't better.

3. mechanical enforcement — if a rule matters, make it mechanical. Prose drifts, checks don't.

4. skills vs. docs — skills are for workflows; docs are for knowledge. The wrong mechanism is worse than no mechanism.

5. earn the check — coverage is cheap, false positives are not. Every mechanical check has to earn its slot or it trains you to ignore failures.

6. errors as remediation — error messages are instructions written for the next agent. If a human (or model) can't act on the message, the check isn't done.

7. fast feedback wins — pre-commit beats CI for the same check. The earlier a violation surfaces, the cheaper it is to fix.

8. immutable decisions — ADRs are append-only. Supersede with a new record; never edit history.

9. sub-agents as firewalls — delegate research and investigation so the implementation context stays clean. Context discipline is the point, not the second agent.

10. watch the loop — the first iterations of any autonomous loop tell you whether your harness holds. The harness fails where you aren't looking.

11. compound the fixes — every agent mistake is a harness improvement waiting to be made. Failure modes should compound, not recur.

12. project as source of truth — not the chat, not the agent, not the prompt. The repo outlives the session, the model, and the conversation.

Part I — Foundations

1. What harness engineering is, and what it isn't

The term *harness engineering* was coined by OpenAI in 2025 to name a discipline that practitioners had been doing informally for two years: building the environment around an AI coding agent so that the agent produces reliable, maintainable work over time.

The framing matters. Earlier discussions of AI-assisted development focused on the *agent* — better models, better prompts, better tools. Harness engineering treats the agent as a given and focuses on everything else: how context is delivered, what constraints the agent operates under, what feedback loops correct its mistakes, how knowledge persists across sessions.

> An agent without a harness is a guess generator. An agent with a harness is a force multiplier.

A working definition:

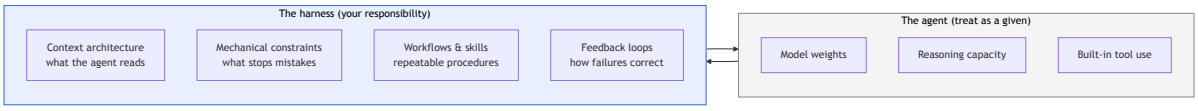
> **Harness engineering** is the discipline of designing the project environment — its documentation, tooling, constraints, and feedback loops — so an AI coding agent can produce correct, maintainable work reliably and at scale.

A few things harness engineering *is not*:

- **It is not prompt engineering.** Prompt engineering tunes a single request. Harness engineering shapes every request the agent will make over the project's lifetime.
- **It is not agent design.** You're not training a model or building tools for general use. You're equipping *your* project so *any* capable agent can work on it well.
- **It is not configuration.** A `CLAUDE.md` and a few linters is the starting point, not the destination. Harness engineering is ongoing: every agent failure is a harness improvement opportunity.
- **It is not a substitute for engineering judgment.** The harness amplifies whatever discipline you bring to it. It does not replace it.

2. The agent vs. the harness: a mental model

The clearest way to internalize harness engineering is to draw a hard line between the *agent* and the *harness*, and notice what falls on each side.



The arrow goes both ways for a reason. The harness *delivers* context to the agent and *receives* the agent's actions. Every part of the harness is something you build, maintain, and improve. None of it is part of the model.

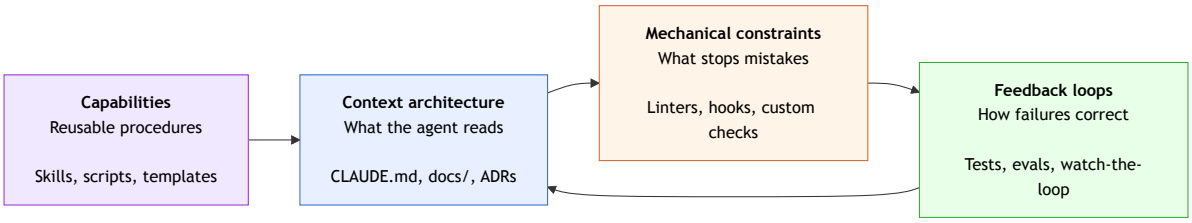
Two consequences fall out of this framing:

The agent's failures are usually your harness's failures. When an agent reformats a file you didn't ask it to, the question isn't "why is this model bad at following instructions?" — it's "where in my harness did the rule against reformatting fail to land?" Often the rule was in prose the agent had to read and remember, not a check that would have failed mechanically. *That's a harness gap.*

You cannot harness-engineer your way out of every problem. If a task genuinely requires reasoning the model cannot perform, no harness fixes it. The harness amplifies; it does not transform. Knowing where the harness ends is as important as knowing where it begins.

3. The four primitives

Every harness, regardless of project size or stack, is built from the same four primitives. They are independent in what they do but tightly coupled in how they reinforce each other.



Context architecture

What the agent reads when starting a task. Includes the engagement file (`CLAUDE.md`, `AGENTS.md`, `.cursor-rules` — names vary), the project knowledge base (`docs/`), Architecture Decision Records, and any reference material the agent may pull in on demand.

Done well: the agent loads exactly what's needed for the current task, no more. Done poorly: the agent loads a monolithic file containing everything and works with a polluted context the rest of the session.

Covered in **Part II**.

Mechanical constraints

The checks that catch mistakes before they reach a human reviewer (or production). Formatters, linters, type checkers, custom project-invariant scripts, test suites — anything that runs automatically and fails when something is wrong.

Done well: each check earns its slot, and its error messages tell the agent how to fix the violation. Done poorly: a graveyard of disabled CI jobs and false-positive-laden linters that everyone (human and agent alike) has learned to ignore.

Covered in **Part III**.

Capabilities

Reusable procedures the agent invokes when a known workflow applies. In Claude Code these are *skills*; in other systems they may be slash commands, custom tools, or scripts. The distinction from context: capabilities are *executed*, not *read*.

Done well: high-frequency workflows are skill-wrapped so each invocation is consistent and traceable. Done poorly: skill sprawl, where every minor task becomes a skill and the metadata budget bloats the agent's startup.

Covered in **Part IV**.

Feedback loops

The mechanism by which the agent (and you) discover when something is wrong and correct it. Includes test failures, eval scores, build errors, and — for autonomous work — the structured research-plan-execute-verify cycle the agent walks each iteration.

Done well: failures are visible early, attributable to a specific cause, and produce harness improvements rather than just patches. Done poorly: long-running agents grind through hundreds of iterations producing plausible-looking code that nobody verifies until much later.

Covered in **Part V**.

Why the diagram has arrows between them

The four primitives reinforce each other in concrete ways:

- **Context → constraints.** Constraints have to reference *something*. An error message that says "see `docs/privacy-constraints.md`" only works if that document exists and is current. The constraint depends on the context.
- **Constraints → loops.** A test failure is only a useful feedback loop if the constraint that failed is clear and actionable. Constraints feed the loops.
- **Capabilities → context.** A skill that wraps a workflow is itself part of the agent's discoverable context. Skills *are* context, just delivered on demand.
- **Loops → context.** Every failure is a candidate harness improvement. Loops feed back into context updates — new ADRs, new rules, new constraints — when the failure mode is worth capturing.

A harness with strong context but weak constraints drifts — the knowledge is right, but nothing stops the agent from acting against it. A harness with strong constraints but weak loops goes stale — the checks are enforced, but failures never feed back into improving them. A harness with strong loops but weak context keeps re-deciding things that were already settled, because the reasoning was never written down. No single primitive compensates for the others; a harness improves only when all four do.

4. What good looks like

The hardest part of this discipline is knowing when you're done — when to stop adding harness and start letting it work. A few signals that a harness is in good shape:

- A fresh agent session orients itself in under one minute by reading the engagement file and one or two routed docs. It does not need you to re-explain the project.
- The agent rarely violates a project invariant, because the harness fails loudly when it does.
- When the agent makes a novel mistake, you can identify which harness layer should have caught it, and fixing that layer takes minutes, not hours.
- The repository alone — without your chat history, without your memory, without you in the room — is enough for someone (or some agent) to continue the project.
- You spend most of your time on the engineering, not on the agent.

A harness that isn't quite there yet looks like the inverse: every session starts with re-explanation, the same mistakes recur, errors require human translation, and the project's coherence lives in your head rather than in the repo. If that describes your current setup, the rest of this guide is for you.

Part II — Context architecture

> *Context is the substrate of every agent task. If it's wrong, nothing downstream can recover.*

The first thing an AI coding agent does on any task is read whatever orientation files the project provides. What lives in those files — and how they're structured — shapes every action the agent takes for the rest of the session.

This Part covers how to organize that material. It is the most important Part in this guide. Get this right and the rest of the harness has solid ground to stand on; get it wrong and no amount of constraint engineering or loop design will compensate.

---

5. The progressive disclosure pattern

Most projects start with a single context file — `CLAUDE.md`, `AGENTS.md`, `.cursorrules`, or whatever the current agent expects. Over time, that file grows. Project background gets added. Architecture notes get appended. Then the tech stack. Then milestone tracking. Then "rules the agent kept breaking." Then notes from the last refactor.

Three months in, the file is 400 lines long. The agent loads all 400 lines on every session, whether the current task is "fix this test" or "rewrite the deployment script." The context the agent works with is dominated by material irrelevant to the current task.

This is the failure mode every harness must avoid. The pattern that prevents it is *progressive disclosure*.

The principle

> Load only what the current task needs. Make everything else available, but not loaded by default.

Progressive disclosure is the same idea that underlies good API design (don't expose every parameter; let advanced ones be opt-in), good UI design (collapse non-essential controls), and good documentation (front-load the basics; link to depth). Applied to agent context, it means:

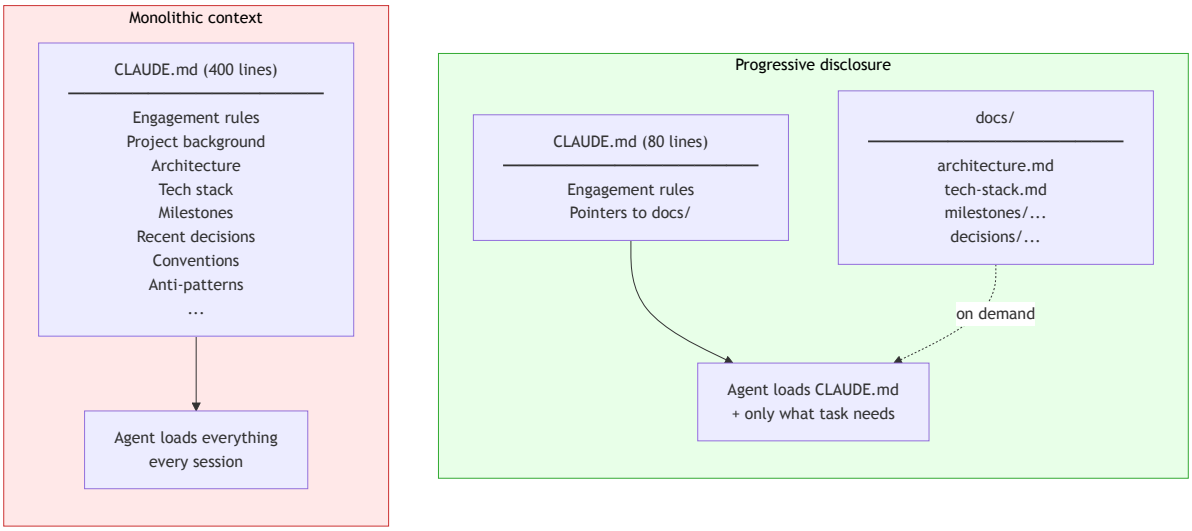
- The always-loaded file contains *only* behavioral rules and a map of what else exists
- Everything else lives in separate files, loaded on demand
- The agent (or you) decides what to read based on the current task

Why context isn't free

It's tempting to think that more context is always safer — more information means the agent is less likely to do something dumb. This is wrong, and the data is now clear:

- **Long contexts degrade reasoning quality.** Empirical work has shown that LLM performance on multi-step reasoning declines noticeably once context utilization exceeds roughly 40% of the model's window. The model is not paying equal attention to all of it.
- **Long contexts dilute attention.** Material that is *relevant* to the current task competes for the model's attention with material that *isn't*. Adding more of the latter hurts the former.
- **Long contexts cost money.** Every token in context is a token billed (and counted against rate limits) on every turn.

The combination means: a 400-line orientation file isn't safer than a 100-line one. It's *worse* on reasoning quality, *worse* on cost, and *worse* on attention to the rules that actually matter.



Practical effect

In practice, progressive disclosure changes the agent's session in three ways:

1. **The engagement file fits on one screen.** ~80–120 lines is a healthy target. Anything longer is probably mixing engagement with knowledge.
2. **The agent learns to route.** When asked about something it doesn't see in the engagement file, it goes to the docs map and reads the relevant doc. This routing behavior is exactly what you want.
3. **Updates are local.** Changing the architecture doesn't touch the engagement file. Adding a milestone doesn't touch the architecture doc. Each file has one job and changes for one reason.

6. System of engagement vs. system of record

The mental model that makes progressive disclosure work is borrowed from enterprise architecture: the distinction between a *system of engagement* and a *system of record*.

	System of engagement	System of record
Question it answers	"How should I behave right now?"	"What is true about this project?"
Content	Behavioral rules, delegation patterns, working style	Architecture, decisions, milestone state, conventions
Lifecycle	Stable; changes rarely	Evolves with the work
Loaded	Every session, always	On demand, by task
Size	Small (~100 lines)	Whatever the material requires
In an agent harness	`CLAUDE.md` / `AGENTS.md`	`docs/`

The two have different *jobs*, different *lifecycles*, and different *audiences* — even when the audience is the same agent.

What belongs in the engagement file

The engagement file should contain only things that change the agent's behavior on *every* task:

- **Identity and scope** — what kind of project this is, in one paragraph
- **Inviolable rules** — constraints that must hold regardless of task (security boundaries, compliance limits)

- **Delegation pattern** — when to hand off to sub-agents
- **Working style** — interaction patterns, how to communicate with the user
- **A routing map** — pointers to where deeper knowledge lives

A useful test for any line you're considering adding: *would removing this line cause the agent to behave differently?* If no, it's knowledge, and it belongs in the system of record.

What belongs in the system of record

Everything else. Architecture diagrams, technology rationale, milestone state, design decisions, glossaries, runbooks, eval methodology, anti-patterns to avoid. Each topic gets its own file. The directory has a `README.md` that serves as the routing table.

The routing table

The most important file in a system of record is its index — usually `docs/README.md`. The agent reads this when the engagement file points it there, and the index decides which deeper file is relevant.

Same concept, different scope

One subtlety worth naming: both the engagement file and the system of record contain an "orientation" section — a description of what the project is. These look like duplicates but serve different purposes.

The engagement file's orientation is a **cold-start primer** — one paragraph, written for an agent that has read nothing. It answers "what am I looking at?" and points to `docs/README.md` for everything else. It should be short enough to read in ten seconds.

The system of record's orientation is an **overview** — two to four paragraphs, written for a contributor actively engaging with the project. It answers "what problem does this solve, who uses it, and what constraints shape it?" It is the foundation the rest of the docs build on.

The distinction matters in practice: a developer writing their first engagement file will be tempted to make the orientation as complete as possible — "more context is safer." That instinct produces a 200-word project description that duplicates `docs/README.md` and inflates the always-loaded context budget. The rule: if a line in the engagement file's orientation could live in `docs/README.md` without loss, it belongs there.

7. Architecture Decision Records

The third piece of context architecture is the *Architecture Decision Record* (ADR) — a short, immutable document recording a single non-obvious design decision and the reasoning behind it.

What an ADR is

An ADR captures four things:

1. **The context.** What situation required a decision?
2. **The decision.** What was chosen?
3. **The alternatives.** What else was considered, and why was it rejected?
4. **The consequences.** What does this commit you to maintaining?

That's it. ADRs are short on purpose. One page is typical; two pages is the upper bound. The point is durable rationale, not exhaustive analysis.

Why ADRs matter for harness engineering

ADRs solve a problem that AI coding agents make worse: *decision evaporation*.

When a human team makes a decision, the reasoning often survives implicitly — in code review comments, in team memory, in shared context. When an AI agent helps make a decision, that implicit memory mostly doesn't form. The decision is captured in the diff; the reasoning is in a chat transcript that is gone the next time someone (or another agent) opens the project.

ADRs make the reasoning *part of the project*. The next session — yours, a teammate's, or a future agent's — can read why something is the way it is. This is the *project as source of truth* principle made concrete.

Numbering and naming

The convention is to number ADRs sequentially with zero-padded filenames:

```
docs/decisions/
├── 0001-use-postgresql.md
├── 0002-event-driven-job-orchestration.md
├── 0003-no-graphql.md
└── README.md
```

Zero-padding (four digits) is standard practice because:

- **Lexicographic sort matches numeric sort.** `0010` sorts after `0009`, whereas `10` would sort between `1` and `2`.
- **Four digits handles any realistic project.** 9,999 ADRs is effectively unlimited.
- **Padding signals identifier, not count.** Leading zeros nudge humans and agents toward treating ADR numbers as stable IDs that should never be reused.

Immutability — the heuristic worth internalizing

> **ADRs are append-only. Supersede with a new record; never edit history.**

This is heuristic #8 from the front page, and it's the rule new ADR practitioners most often violate. The temptation: a decision changes, so go edit the relevant ADR. Don't.

The reason: ADRs are valuable precisely because they record *what was decided, when, and why*. If you edit an ADR to reflect a new decision, you destroy the historical record of *why the original decision made sense at the time*. Six months later, when someone re-encounters the same tradeoff, they have no way to know whether the original reasoning was wrong, the constraints changed, or the team simply forgot.

The right pattern when a decision changes: write a *new* ADR that supersedes the old one. Cross-link them:

```
# 0007 — Move from PostgreSQL to DynamoDB

**Status**: Accepted
**Supersedes**: 0001-use-postgresql.md
**Date**: 2026-08-12

## Context
The constraints have changed: we now need single-digit-millisecond
reads at scale, which PostgreSQL cannot deliver economically...
```

And update the old ADR's status:

```
# 0001 — Use PostgreSQL for primary storage

**Status**: Superseded by 0007-move-to-dynamodb.md
**Date**: 2026-02-04
```

The audit trail of *why the project's design evolved* is now preserved. This is exactly the kind of long-horizon coherence AI-assisted projects need most.

When to write an ADR

Not every decision needs one. ADRs are for decisions that meet at least one of:

- **More than one reasonable alternative existed.** "We picked X over Y because Z."
- **The decision affects multiple files or modules.** Cross-cutting choices benefit from a single discoverable rationale.
- **The decision encodes a constraint** (privacy, performance, compatibility) that isn't visible from any single file.
- **A reviewer would otherwise ask "why?"** ADRs preempt that question.

Don't write ADRs for trivial refactors, naming choices, or decisions that are obvious from the code itself. The bar is *non-obvious* rationale.

8. Anti-pattern: the monolithic context file

Most projects fail at context architecture in the same way: they let the engagement file accumulate everything, then notice the problem too late.

The progression is predictable:

1. **Week 1.** `CLAUDE.md` has 30 lines. Engagement rules, a sentence about the project.
2. **Month 1.** It's 80 lines. A few architecture notes have been added.
3. **Month 3.** 200 lines. Milestone tracking, conventions, a list of common mistakes the agent has made.
4. **Month 6.** 400+ lines. Every architectural decision the team has discussed lives somewhere in the file. Searching it takes longer than reading the relevant code.
5. **Result.** The agent's effective context window for *the actual task* has been halved by orientation material. New sessions still don't have the right context because the most-current information is buried under the most-stale.

The fix is the split this Part describes: keep the engagement file small, move knowledge into a structured directory, route on demand. The refactor is mostly mechanical, takes a few hours, and pays back immediately — every subsequent agent session is faster, cheaper, and more focused.

The principle to internalize:

> **The engagement file is a map. The system of record is the territory. Don't conflate them.**

A few smaller anti-patterns within the monolith family worth naming:

- **Status updates in the engagement file.** "Currently working on Milestone 2, Step 3" belongs in the docs index, not in the always-loaded file. Status is the most volatile content in any harness; putting it in the most-loaded file maximizes drift.
- **Anti-patterns lists in the engagement file.** "Don't use `os.remove`, use `secure_delete`" should be a *mechanical check* (Part III), not a prose rule that the agent might forget. If you find yourself adding "don't do X" lines to your engagement file, that's a signal you need a constraint, not more text.
- **Project history in the engagement file.** "We tried approach A, it didn't work; we tried B, it had issue X..." This is what ADRs are for. Engagement files capture *what to do now*, not *what was tried before*.

What's next

Part II established *where* knowledge lives. Part III addresses *how rules are enforced* — the layer that prevents drift and turns prose-norms into mechanical guarantees.

The transition between them is the heuristic at the center of harness engineering:

> *If a rule matters, make it mechanical. Prose drifts, checks don't.*

That's the subject of Part III.

Part III — Mechanical constraints

If a rule matters, make it mechanical. Prose drifts, checks don't.

Part II established where knowledge lives. This Part addresses what happens when the agent (or you) acts against that knowledge. The answer is not to write a better rule in `CLAUDE.md`. It is to build a check that fails when the rule is violated — automatically, consistently, and with enough information to fix the violation without human translation.

This is the most operationally dense Part in the guide. It includes real configuration and real scripts. The principles are few and simple; the practice takes work to set up and pays back compounding returns once it runs.

---

9. Why prose rules drift, and what to do about it

Every project starts with good intentions encoded in prose. A `CLAUDE.md` or `AGENTS.md` accumulates rules like:

- "Always run `make test` before committing."
- "Don't use `dict` for API responses — use the project's typed response models."
- "New database migrations need a corresponding rollback in `migrations/`."
- "Secrets and API keys must never be hardcoded — use environment variables."

These rules are correct when written. Six weeks later, several of them have been violated — not maliciously, but because:

- The agent had a long context and didn't reach the rule before generating code.
- A new session started without the full orientation file loaded.
- The rule was ambiguous enough that the agent interpreted it differently.
- The rule was buried under thirty other rules and simply not weighted heavily.

This is not a model quality problem. It is a *mechanism* problem. Prose rules are soft — they depend on the agent reading, interpreting, and remembering them correctly on every turn. Mechanical checks are hard — they run regardless of what the agent read, regardless of session length, regardless of interpretation. They cannot be forgotten.

The operative reframe

> Every prose rule you write is a candidate for a mechanical check. The question is not "should I write this rule?" but "is prose the right mechanism for it, or should this be a check?"

Not every rule belongs in a check. Working style rules ("give one suggestion at a time"), delegation patterns ("ask the researcher before benchmarking libraries"), and interaction norms cannot be mechanically enforced — they govern behavior, not artifacts. Those stay in prose.

But rules that govern *artifacts* — what files contain, what code does, what documentation exists — can almost always be made mechanical. The moment you find yourself writing "don't do X in file Y," ask whether a grep, an AST check, or a custom script could catch X automatically.

The compounding return

The economics of mechanical enforcement are asymmetric in your favor:

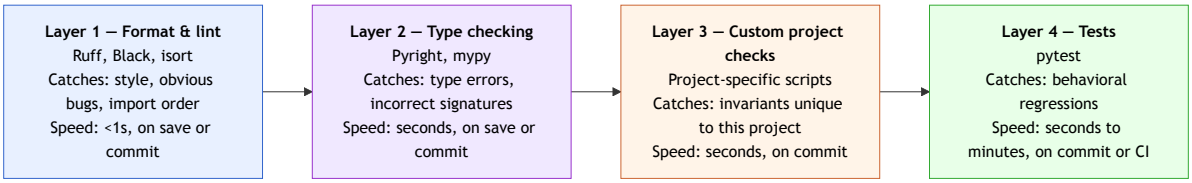
- **Cost of writing a check:** one-time, typically 10–30 minutes for a custom script.
- **Cost of a check catching a violation:** near zero — a failed hook output in the terminal.
- **Cost of a prose rule failing to catch a violation:** debugging time, potential rework, occasional production incident.

A check written today catches the same violation forever — in every commit, by every agent, in every session, without anyone remembering to enforce it. Prose rules require re-enforcement every session. Over the lifetime of a project, five well-written checks outperform fifty prose rules.

---

10. Layered enforcement

Mechanical enforcement is not a single tool — it is a stack of layers, each catching different classes of violation at different speeds.



Each layer is independent. A violation caught at Layer 1 never reaches Layer 4. A violation missed at Layer 3 may or may not be caught at Layer 4 (tests catch behavioral problems, not structural ones). The layers are complementary, not redundant.

Layer 1 — Format and lint (Ruff)

Ruff is the current standard for Python formatting and linting. It replaces Black, flake8, isort, pyupgrade, and several other tools with a single dependency and sub-millisecond execution.

Why this matters for agent-assisted development specifically: agents are not consistent about formatting. Different sessions produce different indentation choices, import orderings, and string quoting styles. Without a formatter, the diff of every agent-produced file is polluted with style noise that obscures real changes. With a formatter, the diff contains only semantic changes.

Minimal `pyproject.toml` configuration:

```toml
[tool.ruff]
target-version = "py311"
line-length = 88

[tool.ruff.lint]
select = [
    "E",    # pycodestyle errors
    "F",    # pyflakes
    "I",    # isort
    "UP",   # pyupgrade
    "B",    # flake8-bugbear
]
ignore = [
    "E501", # line too long — handled by formatter, not linter
]

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
```

Two commands cover all use cases:

```
ruff check .          # lint: report violations
ruff check . --fix    # lint: auto-fix what's fixable
```

```
ruff format .         # format: rewrite files
ruff format . --check # format: report what would change (CI mode)
```

The agent-specific rule: configure Ruff to auto-fix by default in agent sessions. The agent should never have to manually fix a formatting violation — Ruff handles it, the agent moves on. Reserve `--check` mode for CI, where you want failures, not silent auto-fixes.

Layer 2 — Type checking (Pyright)

Pyright (or mypy) catches a class of errors that linters cannot: incorrect types, missing arguments, return type mismatches. These are the errors that don't show up until runtime, and they are disproportionately likely in agent-produced code because agents pattern-match on signatures they've seen in training and occasionally produce plausible-looking-but-wrong invocations.

Minimal `pyproject.toml` configuration for Pyright:

```
[tool.pyright]
pythonVersion = "3.11"
typeCheckingMode = "basic"
include = ["src"]
exclude = ["tests", "migrations"]
```

Start with `"basic"` mode. `"strict"` is valuable but generates enough violations in an existing codebase that it becomes noise before you've addressed the real issues. Graduate to strict incrementally.

Layer 3 — Custom project checks

This is the layer unique to harness engineering. Layers 1 and 2 are generic — any Python project uses them. Layer 3 is where you encode the rules specific to *this* project that no generic tool knows about.

Custom checks are the subject of the next section. The principle here is positioning: Layer 3 runs *after* formatting and type-checking, *before* the full test suite. It is cheap enough to run on every commit, specific enough to catch project-invariant violations, and fast enough not to break developer flow.

Layer 4 — Tests (pytest)

Tests catch behavioral regressions. They are not a substitute for Layers 1–3 — a test suite cannot tell you that a file has the wrong formatting, or that a hardcoded secret was introduced in code that currently has no test coverage. Tests and checks are complementary.

For agent-assisted projects, the test suite has one additional role: it is the agent's primary feedback signal during implementation. An agent that runs tests after each change and reads the failures is operating in a tight feedback loop. An agent that writes code and only runs tests at the end is operating blind. Structure your workflow so the agent runs tests incrementally, not only at completion.

---

11. Pre-commit as the enforcement substrate

The four layers are useless if nobody runs them. The solution is to wire them into git so they run automatically on every commit — no manual invocation, no "did you run the linter?" conversations.

The `pre-commit` framework does this. It manages hook installation, dependency isolation, and per-tool caching in a single configuration file.

Installation

```
pip install pre-commit
pre-commit install   # installs the git hook — run once per repo clone
```

After `pre-commit install`, every `git commit` triggers the configured hooks. If any hook fails, the commit is aborted and the violations are reported. The developer (or agent) fixes them and commits again.

Configuration

The full configuration lives in `.pre-commit-config.yaml` at the repo root. A minimal but complete configuration for a Python project:

```yaml
repos:
  # Layer 1 — Ruff (lint + format)
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.4
    hooks:
      - id: ruff
        args: [--fix]
      - id: ruff-format

  # Layer 2 — Pyright (type checking)
  - repo: local
    hooks:
      - id: pyright
        name: pyright
        entry: pyright
        language: system
        types: [python]
        pass_filenames: false

  # Layer 3 — Custom project checks (examples)
  - repo: local
    hooks:
      - id: check-hardcoded-secrets
        name: Hardcoded secrets
        entry: python tools/check_hardcoded_secrets.py
        language: system
        pass_filenames: false

      - id: check-doc-links
        name: Documentation cross-links
        entry: python tools/check_doc_links.py
        language: system
        pass_filenames: false

  # Layer 4 — Tests (fast unit tests only on commit)
  - repo: local
    hooks:
      - id: pytest-unit
        name: Unit tests
        entry: pytest tests/ --ignore=tests/integration -x -q
        language: system
        pass_filenames: false
```

A few design choices worth noting:

`--fix` on the Ruff lint hook. Auto-fix applies before the commit lands. The developer sees the fix in their working tree, not a failed commit message. This is the right default for agent-assisted work — the agent shouldn't have to iterate on formatting.

Pyright as a `local` hook. Pyright is installed in your project's virtual environment, not managed by pre-commit's isolated environments. This is intentional — you want Pyright to see your actual dependencies, not a stripped-down environment. The tradeoff is that the developer must have Pyright installed locally (via `pip install pyright`).

Integration tests excluded from the commit hook. Integration tests that load real models, hit real APIs, or require specific infrastructure take too long for a commit hook. Run them in CI, not on every local commit. The `--ignore=tests/integration` flag handles this. Fast unit tests on commit; full suite in CI.

`pass_filenames: false` on whole-project checks. Custom checks that validate project-wide invariants (hardcoded secrets, documentation links) need to see the whole project, not just the changed files. Setting this flag tells pre-commit to run the check unconditionally rather than passing it the list of staged files.

Running manually

Pre-commit hooks can be run on demand without committing:

```
pre-commit run --all-files   # run all hooks on every file
pre-commit run ruff          # run a specific hook
```

This is useful when setting up for the first time on an existing codebase — you'll want to see all violations before the first commit rather than discovering them one commit at a time.

The skip escape hatch

Occasionally a commit legitimately needs to skip the hooks — a work-in-progress checkpoint, a merge commit, or an emergency fix. The escape hatch:

```
git commit --no-verify -m "wip: checkpoint"
```

Use this sparingly. The value of pre-commit is in its unconditional enforcement. Every `--no-verify` is a violation that slipped through. If you find yourself reaching for it regularly, the problem is a hook that's too strict or too slow — fix the hook, don't bypass it.

---

12. Writing remediation-grade error messages

This is the detail that separates a functional harness from an excellent one, and it is almost always underinvested.

A check that fails with a cryptic or generic message forces the developer (or agent) to diagnose the problem themselves before fixing it. A check that fails with a remediation-grade message tells them exactly what went wrong, where, and how to fix it. The second check is not just more helpful — it is faster, less error-prone, and critically, *actionable by the agent without human translation*.

The standard to meet

Every custom check error message should answer four questions:

1. **What rule was violated?** Name the invariant, not the symptom.
2. **Where exactly?** File path and line number if applicable.
3. **Why does this rule exist?** One sentence — enough that a developer unfamiliar with the rule understands its purpose.
4. **How do I fix it?** Concrete action, not a vague gesture toward the docs.

A bad error message

```
ERROR: Hardcoded secret detected.
```

This tells the developer that something went wrong. It does not tell them what, where, why, or how to fix it. A human developer will grep for context; an agent will produce a guess.

A remediation-grade error message

```
HARDCODED SECRET VIOLATION
  File: src/services/payments.py, line 23
  Rule: Secrets, API keys, and credentials must never be hardcoded in
        source files. They must be read from environment variables or a
        secrets manager at runtime. Hardcoded secrets are committed to
        version control, logged in plaintext, and cannot be rotated
        without a code change.
  Found: string matching Stripe live key pattern (value redacted)
  Fix:   Replace with: stripe_key = os.environ["STRIPE_SECRET_KEY"]
         Add STRIPE_SECRET_KEY to .env.example (with a placeholder
         value) and to the project's secrets documentation in
         docs/operations.md.
         To allowlist a pattern (e.g. a test fixture using a fake key
         prefix), add it to ALLOWED_PATTERNS in
         tools/check_hardcoded_secrets.py.
  Docs:  docs/security-constraints.md
```

This message tells the agent — in a single read — what rule applies, exactly where the violation is, why the rule exists, two concrete paths to resolution, and where to find more detail. The agent can act on this without asking a follow-up question.

Implementing remediation-grade messages in Python

Custom checks are ordinary Python scripts. The pattern is straightforward:

```python
#!/usr/bin/env python3
"""
check_hardcoded_secrets.py

Scans src/ for string literals that match common secret patterns
(API keys, tokens, credentials). Fails with a remediation message
if any are found outside of explicitly allowed patterns.
"""

import ast
import re
import sys
from pathlib import Path

# Regex patterns that suggest a hardcoded secret
SECRET_PATTERNS = [
    re.compile(r"sk_live_[a-zA-Z0-9]{20,}"),   # Stripe live key
    re.compile(r"ghp_[a-zA-Z0-9]{36}"),          # GitHub personal token
    re.compile(r"AKIA[0-9A-Z]{16}"),             # AWS access key ID
    re.compile(r"-----BEGIN (RSA|EC) PRIVATE KEY-----"),
]

# Literal strings that are explicitly allowed (e.g. fake keys in tests)
ALLOWED_PATTERNS = [
    "sk_test_",     # Stripe test keys are safe to commit
    "example",
```

```
    "placeholder",
    "your-key-here",
]

VIOLATION_MESSAGE = """
HARDCODED SECRET VIOLATION
  File: {file}, line {line}
  Rule: Secrets and credentials must never appear in source files.
        Read them from environment variables or a secrets manager at
        runtime. See docs/security-constraints.md.
  Found: string matching pattern: {pattern} (value redacted)
  Fix:   Replace with an environment variable read:
           value = os.environ["YOUR_SECRET_KEY"]
         Add the variable name with a placeholder to .env.example:
           YOUR_SECRET_KEY=your-key-here   # never commit real values
         Add the real value to your local .env (which is gitignored).
         To allowlist a test fixture, add its prefix to ALLOWED_PATTERNS
         in tools/check_hardcoded_secrets.py.
  Docs:  docs/security-constraints.md
"""


def check_file(path: Path) -> list[str]:
    violations = []
    source = path.read_text()
    try:
        tree = ast.parse(source)
    except SyntaxError:
        return violations

    for node in ast.walk(tree):
        if not isinstance(node, ast.Constant) or not isinstance(node.value, str):
            continue
        value = node.value
        if any(allowed in value for allowed in ALLOWED_PATTERNS):
            continue
        for pattern in SECRET_PATTERNS:
            if pattern.search(value):
                violations.append(
                    VIOLATION_MESSAGE.format(
                        file=path,
                        line=node.lineno,
                        pattern=pattern.pattern,
                    )
                )
                break
    return violations


def main() -> int:
    src_dir = Path("src")
    if not src_dir.exists():
        return 0
```

```
    all_violations = []
    for py_file in src_dir.rglob("*.py"):
        all_violations.extend(check_file(py_file))

    if all_violations:
        for v in all_violations:
            print(v, file=sys.stderr)
        return 1
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

Notice the structure: the violation message is a module-level constant with named placeholders. This makes it easy to read, easy to update, and easy to verify that it answers all four questions before the check ships.

The agent-readable error message as a harness artifact

This reframe is worth internalizing explicitly:

> The error message is not documentation for the developer. It is an instruction for the next agent.

When an agent runs into a failed pre-commit hook, it reads the error output and decides what to do next. A vague error produces a guess. A remediation-grade error produces a correct fix. Writing the message for the agent — not for a human who can look things up — changes what you write and how specific you make it.

---

13. Anti-pattern: noisy checks

The most common way a well-intentioned constraint layer degrades is through noise — checks that fire too often, fire on the wrong things, or produce messages so generic that developers learn to dismiss them.

A noisy check is worse than no check. No check is a gap; a noisy check is a gap *plus* a trained dismissal reflex. Once developers (and agents) learn that a particular check fires on false positives, they stop reading its output — and when it fires on a real violation, that violation slips through anyway.

The five sources of noise

1. Too broad a pattern. A text-based grep for `"password"` will flag comments, log messages, test fixtures, and variable names like `old_password` — none of which are secrets. Prefer AST-based checks that inspect string *values* specifically, not any line that contains the word.

2. No allowlist mechanism. A check with no escape hatch forces developers to fight it on every legitimate exception. The result: they disable the check, not the exception. Every check should have an explicit allowlist (as in the example above) that makes exceptions visible and auditable.

3. Enforcing style, not invariants. A check that catches "too many blank lines between functions" is a formatter's job, not a custom check's. Reserve Layer 3 for invariants that Ruff and Pyright cannot catch. If a generic tool can enforce the rule, use it.

4. Running too slowly. A check that takes 30 seconds on every commit will be bypassed with `--no-verify` within a week. Custom checks should complete in under 5 seconds. If a check is slow, profile it — almost always the bottleneck is file I/O that can be narrowed by filtering to changed files.

5. Firing on generated or vendored code. Auto-generated files, vendored dependencies, and migration files should be excluded from all checks. A failed check on generated code is always a false positive, and the fix is always to add an exclusion — which trains developers to add exclusions reflexively, eroding the check's value on real files.

The discipline to maintain a clean check layer

- **Review check failures in the same session they appear.** A check that fires and is dismissed becomes noise; a check that fires and produces an immediate fix is signal.
- **When a check produces a false positive, fix the check, not just the symptom.** If `check_hardcoded_secrets.py` flags a test fixture using a fake key, add the fixture's prefix to `ALLOWED_PATTERNS` and commit that change explicitly. The allowlist is documentation of deliberate exceptions.
- **Retire checks that haven't fired in six months.** A check that has never caught a real violation is either enforcing a rule that the team naturally follows (in which case the check is noise) or enforcing a rule that no longer applies (in which case the check is wrong). Either way, remove it.
- **Every new check gets a review before merging.** Ask: does this check earn its slot? What is the false-positive rate on the existing codebase? What does the error message tell the next developer?

What's next

Part III established the enforcement layer that gives the context architecture from Part II its teeth. The combination — structured knowledge, mechanical enforcement — covers two of the four primitives.

Part IV turns to the third: *capabilities*. Where context is what the agent reads and constraints are what stops mistakes, capabilities are the reusable procedures that make high-frequency workflows consistent and traceable — and the discipline of knowing when to wrap a workflow in a skill versus when to leave it as a documented process.

Part IV — Agent capabilities (skills)

Skills are for workflows; docs are for knowledge. The wrong mechanism is worse than no mechanism.

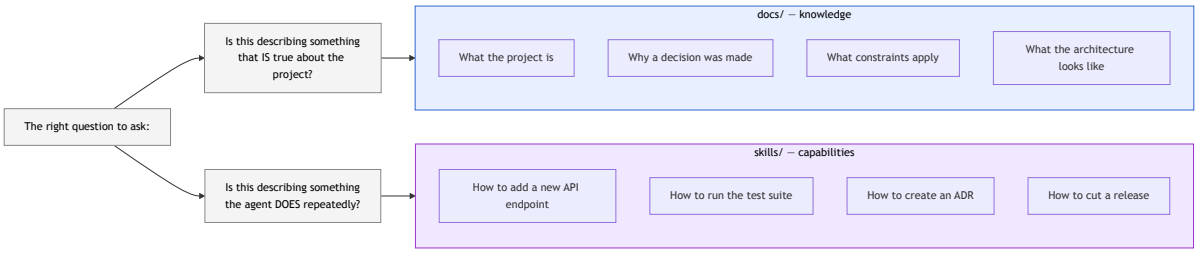
Parts II and III covered what the agent reads (context) and what stops it from acting wrongly (constraints). This Part covers the third primitive: reusable procedures — the workflows the agent executes repeatedly, which deserve to be captured, standardized, and made auto-discoverable rather than re-derived from scratch on every invocation.

In Claude Code, these are called *skills*. Other agent systems call them slash commands, custom tools, or playbooks. The name varies; the concept is the same: a packaged, reusable procedure the agent can invoke consistently without requiring you to re-explain it every session.

---

14. Skills vs. docs: when each is the right tool

The distinction between skills and docs is one of the most commonly confused points in harness engineering, because both use progressive disclosure and both live outside the always-loaded engagement file. The difference is not structural — it is functional.



The operative test

Ask one question about any candidate piece of content:

> **Is this describing something that *is* true about the project, or something the agent *does* repeatedly?**

- "We use PostgreSQL for primary storage" → *is* true → `docs/tech-stack.md`
- "When adding a new database migration, follow these steps..." → *does* repeatedly → a skill
- "All API responses must include a request correlation ID" → *is* true → `docs/architecture.md`
- "When creating an ADR, use this template and update the index" → *does* repeatedly → a skill

The test fails at the edges — some content genuinely straddles both. When in doubt, ask a second question: *would I want the agent to auto-discover this, or do I want to explicitly invoke it?* Auto-discovery is the skill mechanism; explicit reference is the doc mechanism.

Why the wrong mechanism is worse than no mechanism

This is heuristic #4, and the second half — "the wrong mechanism is worse than no mechanism" — is the part that surprises people.

If you put workflow instructions in `docs/`, they don't auto-trigger. The agent reads the doc when you point it there, gets the instructions, and follows them — or doesn't, depending on context pressure. The instructions exist but lack teeth.

If you put project knowledge in a skill, you've created something worse: a skill that auto-triggers on keyword matches, injecting project-knowledge content into unrelated tasks. A skill named "architecture"

with a description broad enough to match "system design" might inject your architecture notes into a session about fixing a failing test — polluting the context with irrelevant material and degrading the agent's focus on the actual task.

The directionality matters:

- **Docs that should be skills** → the agent re-derives the workflow from scratch each time, produces inconsistent results, and you re-explain the same procedure repeatedly. Frustrating, but recoverable.
- **Skills that should be docs** → the agent injects knowledge into wrong contexts, inflates the always-loaded metadata budget, and produces subtly wrong behavior that is hard to diagnose. Harder to recover from.

When uncertain, default to a doc. You can promote to a skill later once the workflow is stable and proven to recur. You cannot easily un-ring the bell of a badly-scoped skill that has been triggering in wrong contexts for two months.

What makes something a good skill candidate

Three conditions, all of which should be true:

1. **High frequency.** The workflow recurs across multiple sessions. A workflow you have done twice may not yet be stable enough to wrap; one you have done ten times has likely revealed its edge cases.
2. **Non-obvious steps.** The workflow has steps that are not derivable from first principles — gotchas, project-specific conventions, ordering constraints that matter. If the agent could reconstruct the procedure correctly from the codebase alone, the skill adds less value.
3. **Consistency matters.** The result should be the same regardless of who (or which agent session) runs it. Database migrations should follow the same structure. API endpoints should follow the same naming and error-handling conventions. ADRs should use the same template. Skills enforce that consistency mechanically.

If all three are true, make it a skill. If only one or two are true, a doc is probably sufficient.

---

15. Writing a skill that auto-triggers correctly

A skill has two distinct parts: the *trigger* (the metadata that tells the agent when to invoke the skill) and the *content* (the instructions the agent follows once invoked). Both need care, but they fail in opposite directions — trigger problems cause wrong invocations; content problems cause wrong executions.

Anatomy of a skill file

In Claude Code, skills live in `.claude/skills/<skill-name>/SKILL.md`. The file follows a specific structure:

```
---
name: add-api-endpoint
description: >
  Use this skill when adding a new REST API endpoint to the service.
  Triggers include: "add an endpoint", "new route", "new API method",
  "expose X via API". This skill covers the full workflow: route
  definition, handler implementation, input validation, error handling,
  and test coverage. Does NOT apply to modifying existing endpoints.
---

# Add a new REST API endpoint

## When to use this skill
```

```
...

## Steps
...

## Gotchas
...

## Verification
...
```

The frontmatter `description` is what the agent reads to decide whether to invoke the skill. The body is what the agent reads after invoking it.

Writing the trigger description

The description has one job: match the right tasks and reject the wrong ones. Four principles:

Be specific about the task class, not the domain. "Use when adding a new REST API endpoint" is specific. "Use for API-related tasks" is too broad — it would match debugging an endpoint, reviewing API documentation, or benchmarking latency, none of which need this skill.

Include trigger phrases explicitly. The agent matches descriptions against the user's request. Listing likely phrasings — "add an endpoint", "new route", "new API method" — increases reliable triggering without broadening the scope.

Exclude adjacent tasks explicitly. If there is a nearby task that might false-trigger, name it: "This skill is for *adding* new endpoints, not for *modifying* existing ones or *documenting* the API." Adjacent exclusions prevent the most common false-positive.

Keep it to 3–5 sentences. The description is read on every session to determine relevance. Long descriptions inflate the always-loaded metadata budget; short ones scan quickly. The description is advertising, not documentation — its job is accurate routing, not complete explanation.

Writing the skill content

Once triggered, the skill body is the agent's procedure. It should read like a senior engineer's handoff note to a capable colleague: complete enough to execute correctly without supervision, specific enough to catch the non-obvious edge cases.

A complete skill body has four sections:

When to use this skill (one short paragraph). Restates the trigger condition in slightly more detail than the description. Helps the agent — and you — verify the skill was invoked correctly before proceeding.

Steps (numbered list). The procedure in order. Each step should be a single action with enough specificity to execute without guessing. Don't collapse two distinct actions into one step.

Gotchas (bulleted list). The non-obvious things that a capable engineer would still get wrong without project-specific knowledge. This is the highest-value section — it is where the skill justifies its existence over a generic doc.

Verification (short list). How to confirm the workflow completed correctly. For a code-producing skill, this is usually "run these tests" and "confirm these files were updated." The verification step closes the loop — the agent knows it is done when verification passes, not when it runs out of steps.

A complete skill example

The following example is for a project that uses FastAPI and follows a specific internal convention for endpoint structure. The gotchas section is where the project-specific knowledge lives — the steps themselves are generic enough that any API project could adapt them.

```
---
name: add-api-endpoint
description: >
  Use this skill when adding a new REST API endpoint to the service.
  Triggers: "add an endpoint for X", "new route", "expose X via API",
  "new API method". Does NOT apply to modifying existing endpoints,
  adding middleware, or documenting the API.
---

# Add a new REST API endpoint

## When to use this skill

When the project needs a new route that does not yet exist. Use this
skill for both simple CRUD endpoints and more complex multi-step
handlers; the structure is the same in both cases.

## Steps

1. Define the route in the appropriate router file under `api/routers/`.
   Group related endpoints in the same router; create a new router file
   only if the resource is genuinely new.

2. Define a Pydantic request model in `api/models/requests.py` and a
   response model in `api/models/responses.py`. Do not use raw dicts
   for request or response bodies.

3. Implement the handler function. Handlers must not contain business
   logic — delegate to a service function in `services/`. The handler
   is responsible only for: parsing the request, calling the service,
   and formatting the response.

4. Add error handling for the expected failure modes. Use the project's
   `raise_http_error()` helper rather than constructing `HTTPException`
   directly — it ensures the error response matches the standard schema.

5. Register the router in `api/main.py` if it is new.

6. Write tests in `tests/api/` covering: the happy path, at least one
   validation error case, and at least one service-layer error case.

## Gotchas

- Route paths use kebab-case, not snake_case. `/user-preferences` not
  `/user_preferences`. The linter catches this, but it is faster to
  get it right the first time.

- The `raise_http_error()` helper expects an error code from
  `api/errors.py`, not a raw HTTP status integer. Using a raw integer
  bypasses the error catalog and breaks the standard error response
  schema.

- Pydantic v2 is in use. Field aliases, validators, and model config
  syntax differ from v1. If you see `@validator` in existing code, that
```

```
  is legacy — use `@field_validator` for new code.

- Tests that hit the database must use the `db_session` fixture, not
  `db` — the latter does not roll back between tests.

## Verification

- `pytest tests/api/ -x` passes with no regressions.
- New tests cover the happy path, one validation error, and one service
  error.
- `ruff check . && ruff format --check .` passes.
- The new endpoint appears in the OpenAPI schema at `/docs`.
```

The skill as a living document

Skills are not static once written. The gotchas section in particular should grow as the team — or the agent — encounters new edge cases. The right cadence: after any session where a skill is used and reveals something non-obvious, add a gotcha. Over time the skill becomes more valuable with each project iteration, accumulating the institutional knowledge that would otherwise evaporate between sessions.

---

16. Anti-pattern: skill sprawl

The opposite failure from under-using skills is over-using them — creating a skill for every task, including ones that do not meet the three conditions for a good skill candidate. The result is *skill sprawl*: a `.claude/skills/` directory with dozens of skill files, most of which trigger rarely or incorrectly, all of which inflate the metadata budget the agent reads on every session.

How sprawl happens

Skill sprawl typically follows a predictable arc:

1. A team discovers skills and finds them powerful.
2. They skill-wrap the obvious high-frequency workflows (good).
3. They skill-wrap some medium-frequency workflows "just in case" (neutral).
4. They skill-wrap one-time setup tasks, research prompts, and reminder notes that belong in docs (bad).
5. The skill directory has 30 files. The agent's startup metadata budget is saturated. Descriptions start to overlap and compete, causing false triggers. The team stops maintaining skills that don't fire, but doesn't remove them because "we might need them."
6. The skills layer produces more noise than signal.

The three signals of sprawl

Descriptions that overlap. If two skills have descriptions that could both trigger on the same user request, one of them is wrong. Either they cover different tasks (sharpen the descriptions) or they cover the same task (merge them).

Skills that have not fired in a month. A skill the agent never invokes is either redundant (the agent handles the task without it), replaced (a different skill covers it), or mis-described (the trigger never matches). Any of these is a reason to retire or fix it.

Skills that contain mostly knowledge, not procedure. A skill file that reads like a doc — lots of "this project uses X because Y" rather than "step 1, step 2, step 3" — is a doc that was placed in the wrong location. Move it to `docs/` and remove the skill.

The maintenance discipline

- **Audit the skill directory quarterly.** Check trigger frequency, description overlap, and content drift.
- **Retire aggressively.** Removing a skill that is not earning its slot is costless — if it turns out to be needed, it can be recreated. Keeping a sprawled skill directory has ongoing costs on every session.
- **One skill per cohesive workflow.** If two procedures always happen together, they are one workflow. If they sometimes happen independently, they are two workflows. Don't split the former; don't merge the latter.
- **Keep descriptions honest.** A description that overpromises — triggering on tasks the skill doesn't actually help with — is worse than no skill. The agent reads it, invokes the skill, and produces a result shaped by content that didn't apply. Update descriptions whenever the scope of a workflow changes.

What's next

Part IV covered the third primitive — capabilities that make high-frequency workflows consistent and traceable. The fourth and final primitive is the feedback loop: the mechanism by which both the agent and the harness itself learn from failures and improve over time.

Part V takes on the discipline of designing loops you can watch, the research-plan-execute-verify cycle, and the question that underlies every autonomous-agent decision: how much autonomy is the right amount, and how do you know when you have crossed the line?

Part V — Feedback loops

The harness fails where you aren't looking.

The first three primitives — context, constraints, and capabilities — are all preventive. They reduce the probability that the agent does something wrong. The fourth primitive, feedback loops, is corrective: it determines how quickly the agent (and the harness itself) discovers and recovers from mistakes when they happen anyway.

A harness without feedback loops is a one-way street. Instructions flow in, code flows out, and errors accumulate silently until a human notices something is wrong. A harness with well-designed feedback loops is a cycle: failures surface quickly, produce actionable signal, and feed improvements back into the harness so the same failure doesn't recur.

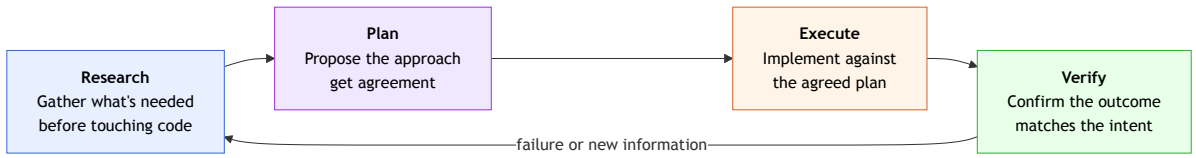
This Part covers the four things that make feedback loops work: a disciplined execution cycle, sub-agent specialization, observable loop design, and a clear-eyed view of the autonomy spectrum.

---

17. Research-plan-execute-verify as a discipline

The most common failure mode in agent-assisted development is not bad code — it is premature code. The agent is asked to implement something, jumps straight to writing files, and produces an implementation that solves the wrong problem, uses the wrong abstraction, or contradicts a decision already captured in the project's ADRs.

The fix is structural: impose a four-phase cycle before any significant implementation begins.



Research

Before writing a line of code, the agent gathers what it needs to make good decisions: the relevant docs from the project's knowledge base, any prior ADRs that constrain the design space, and any external information (library behavior, API contracts, compliance requirements) that affects the approach.

Research is cheap. A session that spends five minutes reading `docs/architecture.md` and two relevant ADRs before coding will produce better output in less total time than one that codes immediately and discovers the constraints mid-implementation.

The research phase also determines whether an investigation should be delegated. Some questions — library comparisons, external API behavior, compliance requirements — are better answered by a specialist sub-agent with a clean context than by the implementation agent pausing to research mid-task. This is covered in the next section.

Plan

Before executing, the agent proposes an approach and gets agreement. The plan does not need to be elaborate — a short description of what will change, what files will be touched, and what the result will look like is enough. The purpose is to surface misunderstandings before they become diffs.

Two things the plan phase catches that execution alone cannot:

- **Wrong problem.** The agent's interpretation of the task differs from what was intended. Catching this in a paragraph costs nothing; catching it after 200 lines of code is expensive.

- **Constraint violations.** The proposed approach contradicts an existing ADR, a mechanical constraint, or a pattern already established elsewhere in the codebase. Reading the plan takes seconds; undoing a committed implementation takes much longer.

In practice, the plan phase is often a single message: "Here's what I propose to do — does this match what you intended?" The value is in the pause, not the formality.

Execute

With research complete and a plan agreed, the agent implements. The execution phase is where most harness engineering effort traditionally focuses — formatters, type checkers, custom checks — and those are necessary. But the research and plan phases upstream make the execute phase dramatically cheaper: the agent is implementing a specific, agreed, constraint-checked plan rather than exploring a solution space.

One discipline that matters specifically during execution: **run checks incrementally, not only at the end.** An agent that runs the test suite after each meaningful change catches regressions immediately and with minimal context. An agent that implements everything and then runs tests at the end faces a complex diagnostic problem when something fails.

Verify

After execution, the agent confirms the outcome matches the intent. Verification has two components:

- **Mechanical verification:** the full check stack passes — formatter, type checker, custom invariants, test suite. This is non-negotiable. An implementation that passes all checks but fails mechanical verification is not complete.
- **Intent verification:** the output actually solves the problem described in the plan phase. Mechanical checks confirm correctness of the implementation; intent verification confirms correctness of the *specification*. These are different checks and both matter.

When verification fails, the loop restarts — not necessarily from scratch, but from the phase where the failure reveals a gap. A test failure sends the agent back to execute. A discovered constraint violation sends it back to plan. New information about the problem sends it back to research.

The loop as a harness artifact

The research-plan-execute-verify cycle is not just a workflow — it is a *harness artifact*. Encoding it in the engagement file (`CLAUDE.md`) means the agent follows it on every significant task without being reminded. Encoding it in a skill means it can be invoked explicitly for tasks where the overhead is warranted.

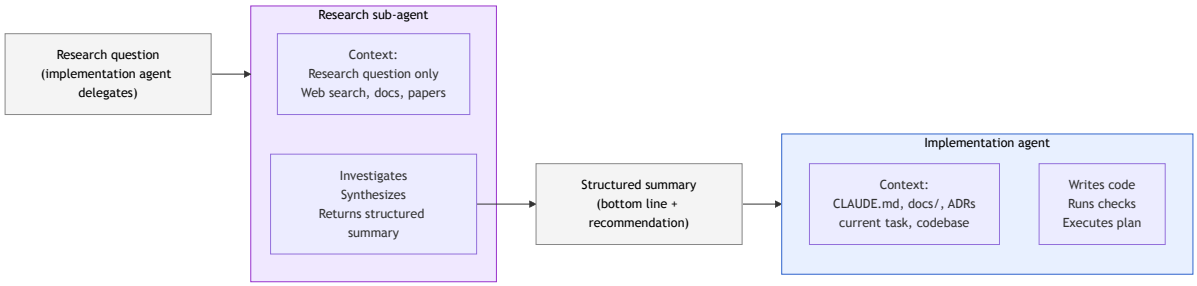
The overhead is worth it. A session that skips the research and plan phases to save time almost always spends that time back at the debugging end of the execute phase, where it costs more.

---

18. Sub-agents as context firewalls

The second mechanism that makes feedback loops work is sub-agent specialization — dividing work between agents with different contexts, different tools, and different scopes.

The key insight is in the framing: sub-agents are not primarily about parallelism or capability. They are about **context isolation**. The implementation agent's context should contain implementation-relevant material. Research, investigation, and synthesis introduce material that is relevant to the research question but irrelevant — and actively distracting — to the implementation task that follows.



What sub-agents isolate

When an implementation agent pauses to research — reading documentation, comparing libraries, investigating external behavior — it pollutes its own context with material that is relevant to the research question and irrelevant to the implementation. This has two costs:

- **Attention dilution.** The model's attention is now split between implementation context and research material. The research material may influence code generation in ways that are subtle and wrong.
- **Context window pressure.** Research output is often verbose. A detailed investigation of three library options, read in full, can consume a substantial fraction of the implementation agent's context window — leaving less room for the codebase, the ADRs, and the actual task.

A sub-agent reads the research question, investigates it in a clean context, and returns a *structured summary* — typically a short document with a bottom line, key findings, and a specific recommendation. The implementation agent reads the summary, not the raw research. The research context stays isolated.

What to delegate and what to handle directly

Not every question warrants a sub-agent. The cost of delegation is coordination overhead — framing the question, waiting for the response, reading the summary. For questions the implementation agent can answer quickly from existing project context, delegation adds latency without adding value.

A simple rule: delegate when the answer requires **going outside the project**. Library comparisons, external API behavior, compliance requirements, benchmark data, security patterns — these require information that isn't in the codebase and would require significant context to gather inline. Handle directly when the answer is in the project's existing docs, ADRs, or code.

Delegate to sub-agent	Handle directly
Comparing two libraries for a use case	Which file contains the database schema
Current compliance requirements for a regulation	What an existing function does
Benchmarking a new model or tool	Whether a test covers a specific case
Security patterns for a new integration	What an ADR decided
Any "how does X work?" about an external system	Debugging a failing test

The structured summary as the handoff artifact

The quality of the sub-agent's output determines the value of the delegation. A research result that dumps raw findings — "I found these five things" — forces the implementation agent to synthesize and decide, which is work that should have happened in the research context. A structured summary that leads with a bottom line and a specific recommendation gives the implementation agent exactly what it needs to proceed.

A minimal effective structure:

```
## Research summary: [topic]
```

```
**Bottom line:** [one sentence — the key finding the implementation agent needs]

**Recommendation:** [specific, actionable, tailored to this project]

**Key findings:**
- Finding 1 (with confidence level or source)
- Finding 2
- ...

**Caveats:** anything uncertain, outdated, or requiring verification
```

The implementation agent reads the bottom line and recommendation. If it needs more detail, it reads the key findings. The raw investigation stays in the research agent's context — it does not travel to the implementation session.

Archiving research

Research summaries are durable. The investigation into "which library should we use for X" is just as relevant six months from now when someone asks the same question. Archiving research output to a `research/` directory at the repo root means future sessions can consult prior investigations rather than re-running them.

This is the *project as source of truth* principle applied to research: the answer to a question the project has already investigated should live in the project, not in someone's memory or a chat transcript that no longer exists.

---

19. Designing loops you can watch

Autonomous loops — where the agent iterates through research-plan-execute-verify without waiting for human input at each step — are the highest-leverage mode of agent-assisted development. They are also the mode where harness failures are most expensive: a poorly designed loop running unattended will compound mistakes across dozens of iterations before anyone notices.

The discipline of loop design is not about limiting autonomy. It is about making loops *observable* — structured so that failures surface quickly, are attributable to a specific cause, and produce harness improvements rather than just patches.

The first-iterations rule

> **Watch the first iterations of any autonomous loop carefully. The harness fails where you aren't looking.**

This is heuristic #10, and the practical implication is specific: when you start a new autonomous loop, do not walk away immediately. Watch the first three to five iterations. You are looking for:

- **Scope drift.** The agent is doing more (or less) than the task specifies. Scope drift caught at iteration two is a plan problem; caught at iteration twenty it is a much larger rework problem.
- **Check failures that the agent is papering over.** An agent that silences a failing test rather than fixing the underlying issue will produce a green check suite that hides broken behavior. This is visible in the first iteration if you're watching; invisible if you're not.
- **Unexpected tool use.** The agent is reading or modifying files outside the expected scope. This is often benign but sometimes indicates a misunderstanding of the task that will compound.
- **Runaway context growth.** Each iteration is loading progressively more context, and the agent's output quality is degrading. This is a loop design problem — the iteration scope is too large, and the loop needs to be restructured around smaller units of work.

If the first few iterations look clean — the agent is doing the right thing, checks are passing, scope is correct — the loop is probably safe to leave. If anything looks off, stop and fix the harness before continuing.

Stop conditions

Every autonomous loop needs an explicit stop condition — a definition of "done" that the agent can evaluate without human input. Without one, the loop either runs indefinitely or stops arbitrarily.

Three types of stop condition, from most to least preferred:

Mechanical: the loop stops when a specific check passes. "Run until the full test suite passes with no failures" is a mechanical stop condition. It is unambiguous, evaluable without human judgment, and naturally aligned with the verification phase of the execution cycle.

Iteration-bounded: the loop stops after N iterations regardless of outcome. "Run for at most ten iterations, then report status." This is a safety net, not a primary stop condition — it prevents runaway loops when the mechanical condition is not being reached. Always combine with a mechanical condition: "run until tests pass, or for at most ten iterations, whichever comes first."

Time-bounded: the loop stops after a wall-clock duration. Less useful than iteration-bounded because iteration cost varies; a ten-minute limit might be two iterations or fifty depending on what the agent is doing. Use time bounds only when iteration count is unpredictable.

Logging for observability

A loop that runs without producing a human-readable trace of what it did is a black box. When it finishes (or fails), you cannot tell which iterations were productive, where it got stuck, or what the harness should have caught.

Minimal useful logging for an autonomous loop:

```
# At the start of each iteration
log.info(f"Iteration {n}: starting — current state: {state_summary}")

# After each check runs
log.info(f"Iteration {n}: {check_name} {'passed' if ok else 'FAILED'}")

# At each decision point
log.info(f"Iteration {n}: decided to {action} because {reason}")

# At loop exit
log.info(f"Loop complete after {n} iterations: {outcome}")
```

The log does not need to be elaborate. It needs to be structured enough that you can read it after the fact and reconstruct what happened. A log that says "iteration 7: tests failed, retrying" is sufficient; a log that says only "working..." is not.

Designing for failure

A well-designed loop treats failure as signal, not just outcome. When an iteration fails — a check does not pass, the output does not match the intent — the loop should:

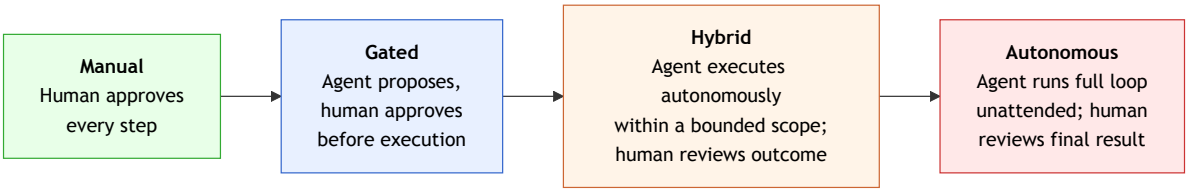
1. **Record the failure specifically.** Not "something went wrong" but "the `test_user_creation` test failed with `AssertionError: expected 201, got 400`."
2. **Pause before retrying.** Immediately retrying the same action that just failed is almost always wrong. The loop should re-enter the research or plan phase with the failure as new information.
3. **Bound the retries.** Three failed attempts at the same fix is a signal that the approach is wrong, not that the agent needs more tries. The loop should escalate — either to a different approach or to human review — rather than continuing to iterate.

A loop that treats failures this way produces a log that is itself a harness improvement guide: every recorded failure is a candidate for a new mechanical check.

---

20. The hybrid autonomy spectrum

Autonomous loops are powerful and expensive — in time, in context budget, and in the cost of mistakes that compound unobserved. The right level of autonomy for a given task is not "as much as possible" — it is "as much as the harness can safely support."



The four levels

Manual — the human approves every individual action before the agent executes it. Appropriate when exploring an unfamiliar codebase, when the task involves irreversible actions (production deployments, data migrations), or when the harness is not yet mature enough to catch the agent's mistakes reliably. Expensive in human time; cheap in mistake cost.

Gated — the agent proposes a plan and the human approves it before execution begins. The execution itself is autonomous within the approved plan. Appropriate for well-scoped tasks in a known codebase with a mature harness. This is the default for design-heavy work: design decisions are made with human input, implementation is autonomous.

Hybrid — the agent runs a bounded autonomous loop on a well-defined task, and the human reviews the outcome. The loop has explicit stop conditions, iteration limits, and sufficient logging that the review is meaningful rather than "it says it worked, I'll take its word for it." Appropriate for tasks that are well-understood, well-constrained, and have strong mechanical verification (a passing test suite is the canonical example).

Autonomous — the agent runs a full loop unattended from problem statement to verified solution. Appropriate only when the harness is mature, the task class is well-understood, the stop condition is mechanical, and the cost of a mistake is low and reversible. Rarely the right choice for novel or design-heavy work; well-suited to routine tasks like dependency updates, test coverage gaps, or documentation generation.

Choosing the right level

The right level is determined by four factors:

Harness maturity. A young harness — one that has not yet been exercised enough to reveal its gaps — does not safely support autonomous operation. Every harness gap that would be caught by a human reviewer in gated mode becomes an unobserved mistake in autonomous mode. Invest in the harness before escalating autonomy.

Task familiarity. A task class the agent has performed many times in this codebase, with a known pattern and known failure modes, is safer to run autonomously than a novel task. The first time the agent implements a new kind of feature, use gated mode. The tenth time, hybrid may be appropriate.

Reversibility. How expensive is a mistake? A mistake in a test file is cheap to reverse — revert the change and try again. A mistake in a database migration script or an infrastructure-as-code file may be expensive or irreversible. Scale autonomy inversely with reversal cost.

Mechanical verification quality. The stronger the mechanical verification — the more completely the check suite can confirm that the output is correct — the safer autonomous operation is. A task with full test coverage and a comprehensive check suite can be verified mechanically. A task where "correct" requires human judgment cannot.

The common mistake: escalating too fast

The most common harness engineering mistake is not choosing too little autonomy — it is escalating to autonomous operation before the harness is ready to support it.

The pattern: a team gets comfortable with gated operation, sees that the agent is performing well, and switches to autonomous loops to save time. The first few loops go well. Then a loop runs into an edge case the harness does not catch, produces incorrect output, and the team discovers the mistake three days later when downstream code fails. The investigation reveals that a mechanical check would have caught the mistake immediately — but nobody had written it yet because the gated operation had been masking the gap.

The discipline: **treat every autonomous loop failure as a harness gap, not an agent failure.** The question after every mistake is not "why did the agent do that?" but "what check should have caught it, and why didn't we have that check?" Every answer is a harness improvement. Over time, the harness becomes mature enough to support broader autonomy safely — but that maturity is earned incrementally, not assumed.

What's next

Part V completed the four primitives: context, constraints, capabilities, and feedback loops. Together they describe a harness that delivers the right information, enforces the right rules, packages the right workflows, and learns from its own failures.

Part VI steps back from the individual primitives to address operating discipline — the practices that keep a harness healthy across sessions, tools, and the inevitable evolution of both the project and the models working on it.

Part VI — Operating discipline

The repo outlives the session, the model, and the conversation.

The five Parts before this one described what to build: context architecture, mechanical constraints, capabilities, and feedback loops. This Part describes how to operate what you've built — the practices that keep a harness healthy across the inevitable churn of sessions, model updates, team changes, and project evolution.

A harness that is well-designed on day one but never maintained degrades into the same monolithic, drift-prone setup it replaced. Operating discipline is what prevents that. It is less glamorous than building the harness, and more important than building it correctly the first time.

---

21. Handoffs between sessions and tools

Every agent session begins with a cold start. The model has no memory of prior sessions; it knows only what the current context provides. This is the most underestimated cost in agent-assisted development — not the per-token price, but the accumulated friction of re-orienting an agent that should already know the project.

A well-designed harness minimises cold-start cost. A poorly designed one externalises it onto the developer, who must re-explain context on every session. The difference compounds: three re-orientations per week across a year is more than 150 sessions spent on setup instead of work.

The two types of handoff

Session handoff — starting a new conversation with the same agent system on the same project. This happens constantly: every time you close a chat, hit a rate limit, or switch from design to implementation mode.

Tool handoff — moving work between different interfaces or agent systems. Design discussion in a chat interface, implementation in an IDE-integrated agent, review in a code-review tool. Each has different strengths; moving between them is inevitable and, without a disciplined handoff, lossy.

What a handoff transfers

The temptation is to summarise the conversation — to replay the reasoning, the false starts, the decisions that were reconsidered. Resist it. A handoff that transfers conversation history transfers noise alongside signal.

What a handoff should transfer is *state*, not *history*:

- **What the project is** — a pointer to the project's orientation docs, not a re-explanation of them. If the docs are current, a single line ("this is the payments-api project — see `docs/README.md`") is enough.
- **What was decided** — references to ADRs that were created or consulted in this session, with a one-line note on why they're relevant to the current task.
- **What the current task is** — the specific, bounded piece of work the next session should continue. Not "we're working on the API" but "implement the `POST /users` endpoint per the plan in `docs/milestones/m2.md`, sections 3 and 4."
- **What's blocking or uncertain** — any open questions or discovered constraints that the next session needs to be aware of before proceeding.

Four items. A well-written handoff is a single short document, typically half a page. Its job is to put a fresh agent session in the position of a capable colleague who just read the relevant docs and is ready to work — not in the position of someone who sat through the whole preceding conversation.

The handoff as a harness artifact

The most effective handoff is not ad-hoc — it follows a template that the team (or a skill) produces consistently. A template-driven handoff has three benefits:

- **It is complete.** A template with four required sections cannot accidentally omit the current task or the blocking questions.
- **It is scannable.** A fresh agent session reads a consistent structure faster than a bespoke summary.
- **It is improvable.** When a handoff fails — the next session still doesn't have the context it needed — the failure is attributable to a specific missing section, and the template is updated.

A minimal handoff template:

```
## Handoff — [date]

**Project:** [name + pointer to docs/README.md]

**Current task:** [specific, bounded description of what to work on next]

**Decisions made this session:**
- [ADR reference or one-line summary of each significant decision]

**Open questions / blockers:**
- [anything the next session needs to resolve before proceeding]

**Engagement rules:** see CLAUDE.md — one step at a time, wait for <<go>>
```

Tool handoffs and the cost of switching

Moving between a chat interface and an IDE-integrated agent is the most common tool handoff. The two have genuinely different strengths:

- **Chat interfaces** are better for design, deliberation, and exploratory discussion. Long context, fast turns, no file-system access required.
- **IDE-integrated agents** are better for implementation, refactoring, and anything that requires reading or writing files. Direct access to the codebase, inline diff review, integrated test runners.

The discipline: **use the right tool for the current phase, and make the handoff explicit.** An implementation task that stays in a chat interface pays the cost of copy-pasting code; a design discussion that migrates into an IDE agent pays the cost of the agent reading files it doesn't need. The handoff document is what makes the switch clean.

The broader principle: **the project docs, not the tool, should be the source of continuity.** A handoff from chat to IDE should work because the IDE agent can read `docs/README.md`, `CLAUDE.md`, and the relevant ADRs — not because the handoff document replayed everything. If the project docs are current, the handoff document can be short. If they're not current, no handoff document is long enough to compensate.

---

22. Keeping the harness evolving instead of drifting

A harness that is not actively maintained drifts. Docs go stale as the codebase changes. Checks that once caught real violations start firing on false positives and get disabled. Skills that once reflected the project's conventions fall out of sync as conventions evolve. The engagement file accumulates new prose rules that should have been mechanical checks.

Drift is not a failure of discipline — it is the natural state of any artifact that is not maintained. The question is not whether drift will happen but whether the team has practices that detect and reverse it before it becomes expensive.

The four drift patterns

Documentation drift — `docs/` describes a system that no longer exists. Architecture docs reference components that were removed; milestone docs claim exit criteria that were quietly abandoned; ADRs describe constraints that were superseded without a new ADR being written.

Detection: the freshness check (Part III) catches files that haven't been touched in a configurable period. The more reliable signal is when a fresh agent session produces output that contradicts the docs — that is always a sign that either the docs or the code is wrong.

Repair: update docs as part of the same commit that changes the code. A commit that changes an architectural component without updating `docs/architecture.md` is incomplete. Make this a team norm, or encode it as a pre-commit check.

Check drift — mechanical checks that were once meaningful have become noise. A check fires on false positives; the team learns to ignore it. A check enforces a pattern that the project has moved away from; it now flags legitimate code. A check was written for a constraint that no longer applies.

Detection: track how often each check fires and what proportion of fires are false positives. A check with a false-positive rate above 10–15% is producing more noise than signal and needs to be fixed or retired.

Repair: fix or retire. A broken check is worse than no check — it trains the team to ignore failures. Retiring it honestly is better than leaving it in place and dismissed.

Skill drift — skills describe workflows that have evolved. A skill that once reflected the team's conventions for adding a new endpoint now produces code that doesn't match how the codebase has developed. The skill is invoked, the agent follows it, and the output is technically correct but stylistically inconsistent.

Detection: when a skill-produced output requires significant post-processing to match the project's current patterns, the skill is drifting. When the gotchas section stops being surprising — when every senior engineer already knows not to do the things it warns against — the skill may have served its purpose and can be simplified or retired.

Repair: update skills in the same session where their gap is discovered. A session that uses a skill and finds it missing a gotcha should add the gotcha before closing. The skill as a living document is only valuable if it is actually updated.

Autonomy drift — the team has gradually escalated to autonomous operation without consciously deciding to, and the harness has not kept pace. Loops that started gated are now running unattended; checks that were adequate for gated operation are not catching the mistakes that autonomous operation introduces.

Detection: autonomous loops producing surprising output, or output that requires significant human correction after the fact. The loop's logging becomes the detection mechanism — a log that shows the agent repeatedly retrying the same failed approach is a signal that the stop condition or the check suite is inadequate.

Repair: step back one level on the autonomy spectrum until the harness gaps are addressed. This is not a failure — it is a calibration. The harness matures by discovering and closing gaps, and that process requires being able to see the gaps.

The update discipline

Four lightweight practices that prevent drift without becoming bureaucratic:

Update docs in the same commit as the code they describe. Never split "change the thing" and "update the docs for the thing" into separate commits. Splitting them means the docs are wrong between the two commits, and "I'll update the docs separately" becomes "I forgot to update the docs."

Write ADRs before implementing, not after. An ADR written after implementation is a post-hoc justification, not a decision record. The value of the ADR is in the plan phase of the execution cycle — the act

of writing it often reveals constraints or alternatives that change the implementation. Written after, it captures what happened but cannot influence it.

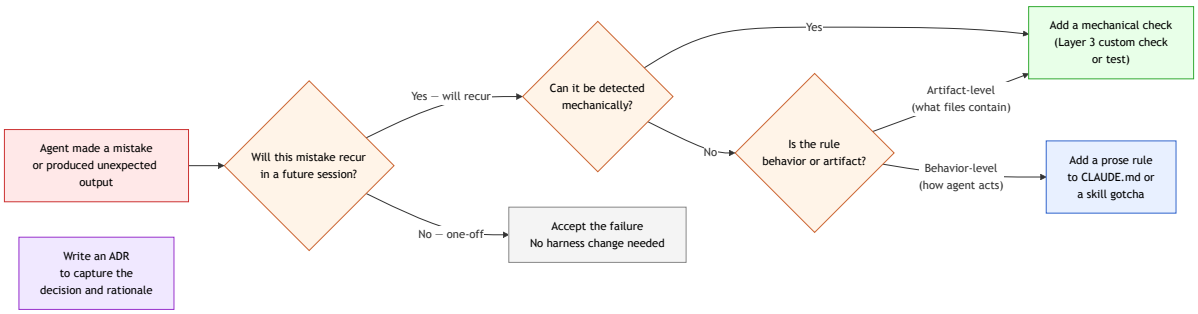
Review the skill directory quarterly. Fifteen minutes per quarter is enough to identify skills that haven't fired, skills with stale gotchas, and skills that have drifted from the current conventions. It is not enough time to let skill sprawl accumulate undetected.

Treat every fresh-session re-explanation as a docs gap. When a new agent session requires you to explain something that should be in the project docs, stop — update the docs, then continue. Every re-explanation is evidence that the system of record is incomplete. Closing that gap immediately costs two minutes; leaving it open costs the same explanation every session.

---

23. When to add a check vs. a rule vs. accept the failure

Not every problem the agent encounters warrants a harness response. Adding a check, a prose rule, or an ADR all have costs — maintenance overhead, false-positive risk, contributor confusion — and those costs must be weighed against the benefit. The discipline is knowing which response, if any, is warranted.



The decision tree in plain language

Will this mistake recur? A mistake that happened because of an unusual task, a transient misunderstanding, or a one-time edge case does not warrant a harness change. Adding a check for every mistake the agent has ever made produces check sprawl; adding a rule for every edge case produces an unreadable engagement file. The bar for a harness change is *recurrence* — the mistake is a predictable consequence of how the task class is structured, not a random error.

Can it be detected mechanically? If yes, a check is almost always the right answer — even if writing the check takes longer than writing a prose rule. A check that catches the mistake on the next occurrence is worth the investment. A prose rule that might be forgotten is not.

Is it behavior or artifact? If the mistake is about how the agent *acts* — the sequence of steps it follows, the questions it asks, the way it communicates — a prose rule in the engagement file or a skill gotcha is the right mechanism. Mechanical checks enforce what files contain; they cannot enforce conversational behavior. If the mistake is about what *artifacts* the agent produces — the content of files, the structure of outputs, the presence or absence of required elements — a check is the right mechanism.

ADRs are for decisions, not mistakes. An ADR is warranted when a mistake reveals that a non-obvious design decision was never documented — so future sessions will encounter the same decision point and lack the reasoning to make it correctly. The ADR captures the decision; the check or rule prevents the mistake. Often both are needed: write the ADR first (to document the decision), then write the check (to enforce it).

Accepting failures

Accepting a failure — making no harness change — is a legitimate outcome of this decision tree, not a failure of discipline. A harness that adds a check for every agent mistake will accumulate technical debt

in the harness itself: overfitted checks, noisy rules, and a skills directory full of gotchas for one-time situations.

The standard for acceptance: the failure was a one-off, caused by conditions that are unlikely to recur, and the cost of a harness change exceeds the expected cost of the failure happening again. These cases exist. Recognising them is part of the discipline.

---

24. The cost picture

Harness engineering is not free, and understanding the cost structure matters for making good decisions about where to invest and how much autonomy to exercise.

Where costs come from

Context costs. Every token in the agent's context window is billed on every turn. A bloated context — from a monolithic engagement file, from research material that should have stayed in a sub-agent, from skills that triggered incorrectly — inflates per-turn cost and degrades output quality simultaneously. Context discipline is not just a performance concern; it is a cost control mechanism.

Autonomous loop costs. An autonomous loop iteration reads context, makes tool calls, produces output, and runs checks — each of which consumes tokens. A loop that runs twenty iterations on a medium-complexity task may consume ten to fifty times the tokens of a single focused manual session. The token cost of a loop is not proportional to the task complexity; it is proportional to the number of iterations, which is determined by how quickly the stop condition is reached.

Mistake costs. A mistake caught by a mechanical check costs the price of the failed commit. A mistake caught by a human reviewer costs review time plus the rework. A mistake that reaches a deployed system costs considerably more. Investing in checks moves cost from the expensive end of this spectrum to the cheap end.

Maintenance costs. Every check, skill, and ADR is a maintenance artifact. It will need to be updated when the project evolves. A harness with fifty checks, thirty skills, and a hundred ADRs has a real ongoing maintenance cost. This is not a reason to avoid building a harness — the maintenance cost of a good harness is lower than the accumulation cost of a bad one — but it is a reason to be selective and to retire artifacts that no longer earn their place.

Rate limits and budget controls

Most agent systems impose rate limits — caps on usage within a rolling window. For autonomous loops especially, rate limits are a real operational constraint. A loop that runs into a rate limit mid-execution must be designed to resume cleanly, not restart from scratch.

Three practices that keep costs predictable:

Monitor usage actively. Most agent platforms expose current usage against the limit. Check it before starting a long autonomous loop; check it mid-loop if the task is significant. The goal is not to stay as far from the limit as possible — it is to never be surprised by a limit when the work is at a critical point.

Budget per task, not per session. Before starting an autonomous loop, estimate the expected token cost and check that the estimate fits within the remaining budget. A loop expected to take fifteen iterations that would consume the entire remaining weekly budget is a loop that should be broken into smaller scoped tasks.

Use cheaper models for routine tasks. Most platforms offer models at different capability and price points. Routine tasks — formatting fixes, documentation updates, dependency bumps, test coverage gaps — do not require the most capable model. Reserving expensive models for tasks that genuinely require strong reasoning is a straightforward cost control practice.

The compounding return on harness investment

The cost picture is asymmetric in one important way: harness investment compounds.

A check written today catches the same mistake in every future session, forever. A skill that packages a workflow makes every future invocation of that workflow cheaper and more consistent. An ADR that captures a design rationale prevents every future session from re-litigating the same decision.

The cost of a harness investment is one-time. The return is per-session, indefinitely. Projects that invest early in harness engineering consistently spend less time on agent management over their lifetime than projects that defer it — not because the harness prevents all mistakes, but because the mistakes it does prevent are the high-frequency, high-cost ones that compound most aggressively without intervention.

What's next

Part VI completes the guide's main content. What remains are the appendices — practical reference material for putting the guide's principles into practice.

- **Appendix A** — Reference toolchain: minimal viable configurations for Ruff, Pyright, pre-commit, and pytest, ready to drop into a new project.
- **Appendix B** — Templates: the engagement file skeleton, `docs/README.md` skeleton, ADR template, skill template, and handoff template referenced throughout the guide.
- **Appendix C** — Further reading: the source material this guide draws on, for readers who want to go deeper on any of the primitives.

Appendix A — Reference toolchain (Python)

This appendix collects the minimal viable configurations for each tool in the harness engineering stack. These are starting points — copy them into a new project and adjust as the project's needs become clear. Every setting included here earns its place; nothing is included for completeness alone.

All configurations assume Python 3.11+. For earlier versions, adjust `target-version` and `pythonVersion` accordingly.

A.1 Project layout assumptions

The configurations below assume this layout:

```
project-root/
├── src/                  # application source code
├── tests/                # unit tests
│   └── integration/      # slow tests that load models or hit external services
├── tools/                # custom check scripts (Layer 3)
├── docs/                 # project knowledge base
│   ├── README.md
│   ├── architecture.md
│   ├── tech-stack.md
│   └── decisions/
├── .claude/
│   └── skills/           # agent skill files
├── .env                  # real secrets — gitignored
├── .env.example          # variable names with placeholders — committed
├── .pre-commit-config.yaml
├── pyproject.toml
├── CLAUDE.md
└── README.md
```

Adjust paths in the configurations below to match your actual layout.

A.2 Ruff (lint + format)

Ruff replaces Black, flake8, isort, and pyupgrade in a single dependency. Add to `pyproject.toml`:

```toml
[tool.ruff]
target-version = "py311"
line-length = 88

[tool.ruff.lint]
select = [
    "E",    # pycodestyle errors
    "F",    # pyflakes (undefined names, unused imports)
    "I",    # isort (import ordering)
    "UP",   # pyupgrade (modernise syntax for target Python version)
    "B",    # flake8-bugbear (likely bugs and design issues)
    "SIM",  # flake8-simplify (unnecessary complexity)
]
ignore = [
    "E501", # line too long — handled by the formatter, not the linter
```

```
]

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["S101"]  # allow assert in tests

[tool.ruff.format]
quote-style = "double"
indent-style = "space"
skip-magic-trailing-comma = false
```

Common commands:

```
ruff check .              # report lint violations
ruff check . --fix        # auto-fix what's fixable
ruff format .             # reformat files
ruff format . --check     # report what would be reformatted (CI mode)
```

Agent sessions: use `--fix` and `ruff format` by default so the agent never has to manually resolve style violations. Reserve `--check` for CI.

A.3 Pyright (type checking)

Add to `pyproject.toml`:

```
[tool.pyright]
pythonVersion = "3.11"
typeCheckingMode = "basic"
include = ["src"]
exclude = [
    "tests",
    "migrations",
    "**/__pycache__",
]
reportMissingImports = true
reportMissingTypeStubs = false   # suppress noise from untyped third-party libs
```

Start with `basic` mode. Strict mode is valuable but generates enough violations in a typical codebase that it becomes noise. Graduate to strict incrementally:

1. Start with `"basic"` — catches the most impactful type errors.
2. Once `basic` is clean, enable `"standard"` — adds more inference.
3. Once `standard` is clean, enable `"strict"` — requires explicit types everywhere.

Installation:

```
pip install pyright
```

Pyright runs as a `local` hook (see pre-commit config below) because it needs to see your project's actual installed dependencies. Keep it in your project's virtual environment, not managed by pre-commit's isolated environments.

A.4 Pre-commit

`.pre-commit-config.yaml` at the repo root:

```
repos:
  # Layer 1 — Ruff: lint and format
```

```yaml
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.4.4   # pin to a specific version; update deliberately
    hooks:
      - id: ruff
        args: [--fix]       # auto-fix before commit
      - id: ruff-format

  # Layer 2 — Pyright: type checking
  - repo: local
    hooks:
      - id: pyright
        name: pyright
        entry: pyright
        language: system
        types: [python]
        pass_filenames: false   # pyright checks the whole project

  # Layer 3 — Custom project invariant checks
  # Add your own check scripts here. See Appendix B for templates.
  - repo: local
    hooks:
      - id: check-hardcoded-secrets
        name: Hardcoded secrets
        entry: python tools/check_hardcoded_secrets.py
        language: system
        pass_filenames: false

      - id: check-doc-links
        name: Documentation cross-links
        entry: python tools/check_doc_links.py
        language: system
        pass_filenames: false

  # Layer 4 — Fast unit tests only
  # Integration tests belong in CI, not in the commit hook.
  - repo: local
    hooks:
      - id: pytest-unit
        name: Unit tests
        entry: pytest tests/ --ignore=tests/integration -x -q
        language: system
        pass_filenames: false
```

Setup (run once per repo clone):

```
pip install pre-commit
pre-commit install
```

Run manually on all files:

```
pre-commit run --all-files
```

Escape hatch (use sparingly):

```
git commit --no-verify -m "wip: checkpoint"
```

Keeping hook versions current: update `rev` values deliberately, not automatically. A hook version update can introduce new violations that break the developer's flow if landed unexpectedly. Review the

changelog before updating.

A.5 pytest

Add to `pyproject.toml`:

```toml
[tool.pytest.ini_options]
pythonpath = ["."]
testpaths = ["tests"]
addopts = [
    "-ra",          # show summary of all non-passing tests
    "--tb=short",   # shorter traceback format
]
filterwarnings = [
    "ignore::DeprecationWarning",
]
```

Test layout convention:

```
tests/
├── conftest.py           # shared fixtures
├── test_users.py         # unit tests for users module
├── test_payments.py      # unit tests for payments module
└── integration/
    ├── conftest.py       # integration-specific fixtures
    └── test_api.py       # tests that hit the real database or network
```

Common commands:

```
pytest tests/ --ignore=tests/integration   # fast unit tests only
pytest tests/                              # all tests including integration
pytest tests/ -x                           # stop on first failure
pytest tests/ -k "test_user"               # run tests matching a pattern
pytest tests/ -v                           # verbose output
```

For agent sessions: run `pytest tests/ --ignore=tests/integration -x -q` by default. The `-x` flag stops on the first failure — this keeps the agent's feedback loop tight, addressing one failure at a time rather than producing a wall of failures that is harder to diagnose.

A.6 Custom check script template

A minimal template for a Layer 3 custom check. Copy this into `tools/check_<name>.py` and fill in the invariant-specific logic:

```python
#!/usr/bin/env python3
"""
check_<name>.py

[One sentence describing what invariant this check enforces and why.]
"""

import sys
from pathlib import Path

# Patterns or values that are explicitly allowed (exceptions to the rule).
```

```
# Each entry should have a comment explaining why it is allowed.
ALLOWED_PATTERNS: list[str] = [
    # "example-allowed-value",   # reason it's allowed
]

VIOLATION_MESSAGE = """
[RULE NAME IN CAPS]
  File: {file}, line {line}
  Rule: [One to two sentences stating the invariant and why it exists.]
  Found: {detail}
  Fix:   [Concrete, actionable fix — specific enough that the agent can
          act without asking a follow-up question.]
         [If there are multiple fix paths, describe each one.]
         To allowlist a legitimate exception, add it to ALLOWED_PATTERNS
         in tools/check_{name}.py.
  Docs:  [path to the relevant doc, e.g. docs/security-constraints.md]
"""


def check_file(path: Path) -> list[str]:
    """Return a list of violation messages for the given file."""
    violations = []
    source = path.read_text()

    # --- Your check logic here ---
    # Prefer AST-based checks over text-based ones for anything nuanced.
    # Text-based checks (grep-style) produce more false positives.

    return violations


def main() -> int:
    src_dir = Path("src")
    if not src_dir.exists():
        return 0

    all_violations: list[str] = []
    for py_file in src_dir.rglob("*.py"):
        all_violations.extend(check_file(py_file))

    if all_violations:
        for v in all_violations:
            print(v, file=sys.stderr)
        return 1

    return 0


if __name__ == "__main__":
    sys.exit(main())
```

The four-question test for every violation message: before shipping a check, verify that its violation message answers:

1. What rule was violated?

2. Where exactly (file and line)?
3. Why does this rule exist?
4. How do I fix it?

If any question is unanswered, the message is not done.

A.7 .gitignore essentials

```
# Python
.venv/
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
*.egg-info/
dist/
build/

# Environment
.env          # real secrets — never commit
.envrc

# Tool caches
.ruff_cache/
.mypy_cache/
.pytest_cache/

# Editor
.vscode/settings.json
.idea/

# OS
.DS_Store
Thumbs.db
```

Note: `.env.example` is **not** in `.gitignore`. It is committed — it documents which environment variables the project requires, with placeholder values. `.env` contains real values and is gitignored.

A.8 Installation order for a new project

When setting up a new project from scratch:

```
# 1. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate    # macOS/Linux
# .venv\Scripts\activate     # Windows

# 2. Install project dependencies
pip install -r requirements.txt

# 3. Install development tools
pip install ruff pyright pre-commit pytest
```

```
# 4. Install the pre-commit hooks
pre-commit install

# 5. Run everything once to verify setup is clean
pre-commit run --all-files
pytest tests/ --ignore=tests/integration

# 6. If all checks pass, make the initial commit
git add .
git commit -m "chore: initial project setup with harness toolchain"
```

If `pre-commit run --all-files` fails on a fresh project, fix the violations before making the first commit. Starting with a clean check suite is significantly easier than inheriting a backlog of pre-existing violations.

Appendix B — Templates

Ready-to-use templates for every harness artifact referenced in this guide. Copy, fill in the bracketed placeholders, and delete the instructional comments before committing.

B.1 Engagement file (CLAUDE.md)

```
# [Project name] — Agent engagement rules

This file tells the agent **how to behave** in this repository.
For **what this project is**, see [`docs/README.md`](docs/README.md).

---

## Orientation: read this first

[One paragraph. What does this project do, who uses it, and what
domain is it in? Enough for a fresh agent session to understand
the context without reading anything else.] For architecture,
technology decisions, milestone status, and past research, see
[`docs/README.md`](docs/README.md).

---

## [!] Inviolable constraints

[List any hard rules the agent must never violate, regardless of
how a request is framed. Typical examples: security boundaries,
data privacy rules, compliance requirements. If none apply, omit
this section — don't invent constraints.]

- [Constraint 1 — state it plainly, in one sentence.]
- [Constraint 2]

Details and rationale live in [`docs/[relevant-doc].md`](docs/).

---

## Agent team

| Agent | Role | Model | Invocation |
|---|---|---|---|
| **Coding agent** (default) | Implements, scaffolds, debugs, refactors | [model] | Default |
| **Researcher** (sub-agent) | Investigates, synthesises, advises | [model] | `@researcher` |

### Delegate to `@researcher` before acting on:

- Comparing libraries, tools, or frameworks
- Looking up external service details, quotas, or SDK behaviour
- Investigating compliance or regulatory requirements
- Benchmarking or evaluating models or tools
- Any "how does X work?" question about an external system
```

```
### Handle directly (no delegation needed):

- Writing, editing, and refactoring code
- Running tests and interpreting output
- Debugging errors in existing code
- Implementing a design that has already been researched and decided

---

## Working style

1. **Give one instruction or suggestion at a time, then stop and wait.**
2. Do not proceed until the developer says `<<go>>`.
3. Explain what you are about to do and **why** before doing it.
4. If a decision needs to be made, present options and recommend one —
   then wait for `<<go>>` before implementing.
5. Keep code minimal — prototype quality unless told otherwise.

---
```

B.2 docs/README.md (routing table)

```
# [Project name] — Documentation

This directory is the **system of record** for [project name].
[`CLAUDE.md`](../CLAUDE.md) at the repo root points here for all
project knowledge.

---

## Overview

[Two to four paragraphs. What is this project? What problem does it
solve? Who uses it? What are the key constraints or requirements that
shape the architecture? This is the first thing a new contributor —
human or agent — reads to understand the project.]

---

## Current status

> **[Milestone name]** — [status: in progress / not started / complete]
> **Active focus:** [one sentence on what's being worked on right now]
>
> Update this section whenever the active focus shifts. Keep it to
> three lines maximum.

---

## Read this doc when your task involves…

| If the task is about… | Read… |
```

```
|---|---|
| Overall architecture, data flow, component boundaries | [`architecture.md`](architecture.md) |
| Technology choices, dependency rationale | [`tech-stack.md`](tech-stack.md) |
| [Add rows for your project's docs] | |
| Why a specific design decision was made | [`decisions/`](decisions/) |
| Past research investigations | [`/research/`](../research/) |

---

## Documentation conventions

- **Top-level docs** describe stable, long-lived project properties.
- **`milestones/`** tracks scoped goals and exit criteria per milestone.
- **`decisions/`** holds Architecture Decision Records — immutable
  records of non-obvious design choices.
- **`/research/`** at the repo root holds investigation outputs from
  the researcher sub-agent.

### Update discipline

- Update the **Current status** section whenever focus shifts.
- Update **milestone docs** when scope changes or criteria are met.
- Never edit ADRs — supersede them with new ones if decisions change.
- Significant changes to `architecture.md` or `tech-stack.md` warrant
  an ADR explaining why.
```

B.3 Architecture Decision Record (ADR)

Save as `docs/decisions/NNNN-short-slug.md`:

```
# NNNN — [Short title in imperative form]

**Status:** Proposed | Accepted | Superseded by [NNNN-slug]
**Date:** YYYY-MM-DD
[**Supersedes:** NNNN-slug]   $\leftarrow$ include only if this replaces a prior ADR

---

## Context

[What is the situation that requires a decision? What constraints,
prior work, or external factors are relevant? Describe the problem,
not the solution. One to three paragraphs.]

## Decision

[What was chosen? State it plainly in one paragraph. No justification
here — that goes in Consequences.]

## Alternatives considered

- **[Alternative A]** — [why it was rejected, one to two sentences]
- **[Alternative B]** — [why it was rejected]
```

```
## Consequences

- **Positive:** [what gets easier, more correct, or newly possible]
- **Negative:** [what gets harder, what must be maintained]
- **Neutral:** [tradeoffs that are not clearly wins or losses]

## References

[Links to research outputs, PRs, external docs, or code that informed
the decision. Omit if none.]
```

Numbering: use four-digit zero-padded sequential numbers (`0001`, `0002`, ...). Numbers are never reused, even after supersession.

Index: every ADR must be listed in `docs/decisions/README.md` before it is merged. The index entry format:

```
| 0001 | [Title](0001-slug.md) | Accepted |
```

B.4 Skill file

Save as `.claude/skills/<name>/SKILL.md`:

```
---
name: [kebab-case-name]
description: >
  Use this skill when [specific task class — one sentence].
  Triggers include: "[trigger phrase 1]", "[trigger phrase 2]",
  "[trigger phrase 3]".
  Does NOT apply to [adjacent task that might false-trigger].
---

# [Skill title]

## When to use this skill

[One paragraph restating the trigger condition in slightly more detail
than the description. The agent reads this to confirm the skill was
invoked correctly before proceeding.]

## Steps

1. [First action — specific enough to execute without guessing]
2. [Second action]
3. [Continue as needed — each step is a single action]

## Gotchas

- [Non-obvious thing a capable engineer would still get wrong
  without project-specific knowledge]
- [Another gotcha — add these as you discover them in practice]

## Verification

- [How to confirm the workflow completed correctly]
```

```
- [Usually: run these tests, check these files were updated]
```

Trigger description checklist before shipping:

- ☐ Specific about the task class (not the domain)
- ☐ Includes two to four explicit trigger phrases
- ☐ Excludes the most likely adjacent false-trigger
- ☐ Three to five sentences maximum

Content checklist:

- ☐ When-to-use section confirms the trigger
- ☐ Steps are numbered and each is a single action
- ☐ Gotchas section exists (even if short — this is the high-value section)
- ☐ Verification section closes the loop

B.5 Handoff document

Use when transitioning between sessions or between tools (e.g. chat to IDE):

```
## Handoff — [YYYY-MM-DD]

**Project:** [name] — see [`docs/README.md`](docs/README.md) for orientation.

**Current task:**
[Specific, bounded description of what to work on next. Not "we're
working on the API" but "implement the POST /users endpoint as
described in docs/milestones/m2.md sections 3 and 4."]

**Decisions made this session:**
- [ADR reference or one-line summary of each significant decision]
- [If no decisions were made, write "none — existing ADRs apply"]

**Open questions / blockers:**
- [Anything the next session needs to resolve before proceeding]
- [If none, write "none — proceed with the current task"]

**Engagement rules:** see [`CLAUDE.md`](CLAUDE.md) — one step at a
time, wait for `<<go>>` before proceeding.
```

What to include and exclude:

Include	Exclude
The specific next task	The conversation history
ADR references (pointer, not content)	Reasoning that led to decisions
Open questions	Closed questions
Pointers to relevant docs	Re-explanation of doc content

The handoff should be readable in under two minutes. If it is longer, it is carrying history rather than state.

B.6 docs/decisions/README.md (ADR index)

```
# Architecture Decision Records

ADRs are short, immutable records of non-obvious design decisions.
Each captures the context, the decision, the alternatives considered,
and the consequences.

## When to write one

Write an ADR when:
- More than one reasonable alternative existed
- The decision affects multiple files or modules
- The decision encodes a constraint not visible from any single file
- A future contributor would otherwise ask "why is it done this way?"

## Rules

- **Immutable:** never edit an ADR after it is accepted. If a decision
  changes, write a new ADR that supersedes the old one.
- **Indexed:** every ADR in this directory must appear in the table
  below before merging.
- **Numbered:** four-digit zero-padded sequence (`0001`, `0002`, …).
  Numbers are never reused.

## Template

See [`TEMPLATE.md`](TEMPLATE.md).

## Index

| # | Title | Status |
|---|---|---|
| [0001](0001-example.md) | [Example decision title] | Accepted |
```

Appendix C — Further reading

This guide distils material from several sources. The list below identifies the most useful ones for readers who want to go deeper on any of the primitives.

Annotations describe what each source specifically contributes — not a general summary, but what it adds beyond what this guide covers.

Primary sources

OpenAI — "Harness Engineering: Leveraging Codex in an Agent-First World" The post that named the discipline. Covers the AGENTS.md-as-table-of-contents pattern, the argument for mechanical enforcement over prose rules, and the insight that error messages should double as remediation instructions. The guide you are reading expands on these ideas; the original post is the most concise statement of them.

Geoffrey Huntley — "Everything is a Ralph Loop" The practitioner's account of running single-agent autonomous loops in production. The most useful contribution is the framing that every agent failure is a harness bug, not a model failure — and the discipline of fixing the harness rather than patching around the agent. Also the clearest articulation of why full autonomy is the wrong default for most projects.

Michael Nygard — "Documenting Architecture Decisions" The original ADR proposal from 2011. Short — about 500 words. Establishes the format (context, decision, consequences) and the immutability principle. Worth reading in full; everything written about ADRs since traces back to this post.

Context architecture

Simon Willison — "Context Engineering" Willison's post on the discipline of carefully constructing LLM context windows — what goes in, what stays out, and why it matters more than prompt wording. Directly relevant to the progressive disclosure argument in Part II.

awesome-harness-engineering — community resource list A community-maintained index of harness engineering resources: tools, patterns, case studies, and practitioner accounts. Useful for staying current as the field evolves; the guide you are reading reflects the state of the discipline as of mid-2026, and the field is moving quickly.

Mechanical constraints

pre-commit documentation The authoritative reference for hook configuration, supported languages, and the hook lifecycle. The guide covers the essential configuration; the pre-commit docs cover edge cases, multi-language hooks, and CI integration in more depth.

Ruff documentation The full rule reference, configuration options, and migration guides from Black, flake8, and isort. When the minimal configuration in Appendix A is not enough, the Ruff docs are the right next stop.

Charlie Marsh — "Python tooling could be much, much faster" The essay that motivated Ruff's creation. The core argument: when linting happens in under a second, developers add it to commit hooks and run it constantly; when it takes 20 seconds, they skip it. Speed is a correctness property, not a comfort one.

Agent capabilities and skills

Anthropic — Claude Code documentation The authoritative reference for Claude Code's skill system, CLAUDE.md format, sub-agent invocation, and tool configuration. The guide's treatment of skills reflects the Claude Code implementation; other agent systems implement the same concept differently.

Feedback loops

The Twelve-Factor App Not directly about agent harnesses, but the discipline of designing systems that are auditable, reproducible, and observable transfers directly. The sections on configuration, logging, and process isolation are particularly relevant to autonomous loop design.

Andrej Karpathy — "Software 2.0" The essay that introduced the framing of neural networks as a new kind of software. Relevant background for understanding why harness engineering matters: if models are a new kind of software component, they need the same infrastructure discipline as any other component — versioning, testing, observability, and constraints.

Operating discipline and cost management

Anthropic — API pricing and rate limits Current per-token rates, rate limits, and model tier comparisons. The guide's cost discussion reflects mid-2026 pricing; consult the current docs before making budget decisions, as pricing changes frequently.

Martin Fowler — "Patterns of Enterprise Application Architecture" Not about AI agents — about software architecture patterns that have proven durable across decades. The system-of-record / system-of-engagement distinction (Part II), the argument for mechanical enforcement over convention (Part III), and the role of feedback loops in system health (Part V) all have antecedents in Fowler's taxonomy. Useful for practitioners who want to understand why these patterns are durable, not just how to apply them.

For practitioners building harnesses at scale

Accelerate — Forsgren, Humble, Kim Research-backed analysis of what distinguishes high-performing software delivery teams. The finding most relevant to harness engineering: the practices that most strongly predict delivery performance are automated testing, trunk-based development, and working in small batches — all of which map directly to the mechanical constraint and feedback loop primitives in this guide.

The DevOps Handbook — Kim, Humble, Debois, Willis The practitioner's companion to Accelerate. The chapters on feedback loops, continuous integration, and blameless postmortems translate directly to the harness engineering context. Particularly relevant to Section 19 (designing loops you can watch) and Section 23 (when to add a check vs. accept a failure).